

Self-Reflection and Failure Recovery in Agentic AI Coding Systems: An Empirical Evaluation of Blind Retry, Reflection, and Diagnosis-Based Recovery Strategies

M.Tech Dissertation

Submitted in
partial fulfillment of the requirement
for the degree of

MASTER OF TECHNOLOGY

in
Artificial Intelligence and Data Science

by

Lagad Nakul Kiran

2503032340022

Under the supervision of

Ramizraja Shethwala, Assistant Professor

Dr.Mohammad Arif, Professor



Parul[®]
University

NAAC A++

October 2026

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE PARUL INSTITUTE OF
ENGINEERING & TECHNOLOGY FACULTY OF ENGINEERING & TECHNOLOGY
PARUL UNIVERSITY**

P.O. Limda – 391 760, GUJARAT, INDIA

CERTIFICATE

This is to certify that the work contained in this dissertation thesis entitled “Self-Reflection and Failure Recovery in Agentic AI Coding Systems: An Empirical Evaluation of Blind Retry, Reflection, and Diagnosis-Based Recovery Strategies” submitted by the student, Nakul Lagad, Enrollment No. 2503032340022, studying at Parul Institute of Engineering & Technology for the Dissertation Phase-I / Dissertation Phase-II (M.Tech – Artificial Intelligence and Data Science) is based on his own work carried out under our supervision and that this work/thesis has not been submitted elsewhere for any degree/diploma. To our satisfaction, this work is approved for the Dissertation Phase I / Dissertation Phase II (M.Tech – Artificial Intelligence and Data Science) Internal/External exam.

The extent of plagiarism does not exceed the permissible limit laid down by the University.

Ramizraja Shethwala

Dr. Mohammad Arif

Faculty Supervisor(s)

Date:

Name

HoD, Dept, PIET

Name

Department PG Coordinator

Name

Principal, PIET

THESIS APPROVAL CERTIFICATE

This is to certify that research work embodied in this dissertation thesis entitled “Self-Reflection and Failure Recovery in Agentic AI Coding Systems: An Empirical Evaluation of Blind Retry, Reflection, and Diagnosis-Based Recovery Strategies” carried out by Mr. Nakul Lagad (Enrollment No. 2503032340022) at Parul Institute of Engineering & Technology, is approved for the degree of M.Tech with specialization in Artificial Intelligence and Data Science by Parul University.

Date:

Place: Vadodara, Gujarat

External Examiners’ Sign and Name:

1)

2)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisors, Mr. Ramizraja Shethwala, Assistant Professor, and Dr. Mohammad Arif, Professor, Department of Artificial Intelligence and Data Science, Parul Institute of Engineering & Technology, for their invaluable guidance, encouragement, and scientific and theoretical insight throughout the course of this dissertation. Their constructive feedback at every stage — from the systematic literature review to the design and execution of the empirical study — was instrumental in shaping this work.

I am grateful to the Head of the Department and the senior faculty members of the Department of Artificial Intelligence and Data Science for their constant support and for providing the academic environment necessary to carry out this research.

I would also like to acknowledge the broader institutional support extended by Parul Institute of Engineering & Technology and Parul University in providing access to research resources and infrastructure.

Finally, I acknowledge the open-source community and the maintainers of the SWEbench benchmark, whose publicly available datasets and tools made the empirical portion of this dissertation possible.

Lagad Nakul Kiran
Enrollment No. 2503032340022

PAPER PUBLICATION CERTIFICATE

This is to certify that research work embodied in this dissertation thesis entitled “Self-Reflection and Failure Recovery in Agentic AI Coding Systems: An Empirical Evaluation of Blind Retry, Reflection, and Diagnosis-Based Recovery Strategies” carried out by Mr. Nakul Lagad (Enrollment No. 2503032340022) at Parul Institute of Engineering & Technology for partial fulfillment of M.Tech degree to be awarded by Parul University, has submitted the article entitled “Self-Reflection and Failure Recovery in Agentic AI Coding Systems: A Systematic Review” for publication (Manuscript No. INFSOF-D-26-01742) in Information and Software Technology, published by Elsevier, under Manuscript No. INFSOF-D-26-01742. At the time of preparation of this certificate, the manuscript is identified by the above Elsevier manuscript number. Volume, article number, DOI, and final publication details will be added upon acceptance and publication by the journal. A second article based on the original empirical work carried out during Dissertation Phase-I and Phase-II, comparing **Blind Retry, Reflection-only, and Diagnose+Revise/Diagnosis-based recovery strategies for LLM coding agents**, is **under preparation** and will be submitted to a suitable peer-reviewed software engineering/AI venue after completion of the empirical evaluation.

Date:

Place: Vadodara, Gujarat

Signature and Name of Student

Signature and Name of Faculty Supervisor

ABSTRACT

Autonomous coding agents powered by Large Language Models (LLMs) are increasingly capable of navigating software repositories, generating code, and executing multi-step software engineering tasks. However, reliable recovery from execution failures remains a significant challenge. A systematic review of 35 studies published between 2023 and 2026, conducted as the foundation of this dissertation, identified a specific research gap: although existing studies investigate agent failures, self-reflection, diagnosis, and automated repair, limited empirical evidence directly evaluates whether diagnosis-based recovery improves the resolution of repository-level coding failures compared with simpler retry strategies.

To address this gap, an empirical evaluation pipeline was designed and implemented using real repository-level bugs from the SWE-bench Lite benchmark. Three recovery strategies were evaluated: Blind Retry, Reflection-only, and Diagnose+Revise. A 20-task pilot evaluation showed that 60% of tasks were resolved on the initial attempt, while the remaining failures comprised both genuine logic failures and patch-formatting failures. To ensure that recovery performance was not confounded by patch-generation reliability, the initial unified-diff mechanism was replaced with a SEARCH/REPLACE representation and programmatically generated patches, enabling formatting failures to be separated from reasoning failures.

Following this methodological correction, the three recovery strategies produced identical outcomes in every cell that was evaluated, across two independent evaluation rounds; 16 of the 24 cells defined by the comparison were evaluated, the remainder being constrained by the free-tier request ceiling. None of the four genuine logic failures was successfully recovered, including under the Diagnose+Revise condition, which was provided with fresh test-failure evidence. The results therefore indicate that, within the evaluated setting, providing explicit diagnostic information alone was insufficient to achieve recovery from the observed repository-level logic failures.

The dissertation contributes a fully scripted and auditable evaluation pipeline for studying failure recovery in coding agents, an empirical separation of patch-formatting reliability from reasoning correctness, and a robustness-checked null finding concerning single-round diagnosis-based recovery. The findings motivate future investigation of multi-round recovery, richer repository context, persistent failure memory, and more adaptive planning mechanisms.

Keywords: Agentic AI, Large Language Models, Autonomous Coding Agents, Failure Recovery, Self-Reflection, Repository-Level Software Engineering, SWE-bench, Automated Program Repair

TABLE OF CONTENTS

Contents

CERTIFICATE	2
THESIS APPROVAL CERTIFICATE	3
ACKNOWLEDGEMENT	4
PAPER PUBLICATION CERTIFICATE	5
ABSTRACT.....	6
LIST OF ABBREVIATIONS.....	12
LIST OF FIGURES	13
LIST OF TABLES	14
INTRODUCTION	15
1.1 Background	15
1.2 Evolution of LLM-Based Coding Systems	16
1.3 Autonomous Coding Agents.....	17
1.4 Failure in Repository-Level Coding.....	19
1.5 Self-Reflection and Failure Recovery	20
1.6 Motivation	22
1.7 Problem Statement	23
1.8 Research Questions.....	24
1.9 Research Contributions.....	25
1.10 Scope of the Dissertation	26
1.11 Organization of the Dissertation.....	27
AIM, OBJECTIVES AND RESEARCH SCOPE.....	29
2.1 Aim of the Research.....	29
2.2 Research Objectives.....	29
2.3 Research Questions.....	31
2.4 Scope of the Study.....	32

2.5 Research Assumptions	34
2.6 Limitations of the Study	34
2.7 Chapter Summary	36
LITERATURE REVIEW.....	37
3.1 Overview	37
3.2 Autonomous Coding Agents	38
3.3 Self-Reflection and Iterative Reasoning	40
3.4 Memory Mechanisms in LLM Agents	41
3.5 Planning and Tool Use	43
3.6 Failure Detection and Failure Analysis	43
3.7 Automated Debugging and Program Repair	45
3.8 Evaluation Benchmarks for Coding Agents	46
3.9 Comparative Analysis of the Reviewed Studies	47
3.10 Research Gap Identification	52
3.11 Positioning of the Present Study	55
3.12 Chapter Summary	56
RESEARCH METHODOLOGY	58
4.1 Experimental Overview	58
4.2 Research Questions and Evaluation Design	60
4.3 Dataset and Benchmark	61
4.4 Task Selection	62
4.5 Model and Access Configuration	64
4.6 Repository Context and Task Preparation	65
4.7 Oracle-Based File Localization	67
4.8 Patch Representation and Application	68
4.9 Evaluation Infrastructure	71
4.10 Baseline Evaluation	72

4.11 Recovery Strategy 1: Blind Retry.....	73
4.12 Recovery Strategy 2: Reflection-only.....	73
4.13 Recovery Strategy 3: Diagnose+Revise.....	74
4.14 Metrics and Outcome Classification	76
4.15 The Fairness Retrofit as an Experimental Control	78
4.16 Second Recovery Round.....	80
4.17 Reproducibility and Experimental Controls	81
4.18 Ethical and Practical Considerations.....	82
4.19 Chapter Summary	83
RESULTS AND DISCUSSION.....	85
5.1 Baseline Results.....	85
5.2 Baseline Failure Classification.....	87
5.3 The Patch-Formatting Confound	88
5.4 Results After the Fairness Retrofit.....	89
5.5 Recovery Outcomes: Round One	90
5.6 Recovery Outcomes: Round Two	91
5.7 Cross-Strategy Comparison.....	93
5.8 Interpretation of the Null Result	96
5.9 Patch Reliability and Reasoning Correctness	98
5.10 Methodological Findings	99
5.11 Comparison With the Existing Literature	100
5.12 Threats to Validity	102
5.13 Chapter Summary	103
CONCLUSIONS AND SUMMARY	105
6.1 Summary of Work Carried Out	105
6.2 Conclusions Against the Research Questions.....	105
6.3 Key Conclusions.....	107

6.4 Contributions of the Dissertation	108
6.5 Limitations of the Study	108
6.6 Scope for Future Work	109
6.7 Concluding Remarks	111
REFERENCES	113

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
SWE-bench	Software Engineering Benchmark
WSL	Windows Subsystem for Linux
CRLF	Carriage Return Line Feed
LF	Line Feed
APR	Automated Program Repair
RCA	Root Cause Analysis
JSON	JavaScript Object Notation
CLI	Command Line Interface

LIST OF FIGURES

Figure No.	Title	Page Number
1.1	Autonomous Coding Agent Workflow	11
1.2	Failure and Recovery Strategies	20
1.3	Information Available During Failure Recovery	21
3.1	General Workflow of an Autonomous Coding Agent	38
3.2	Failure Categories in Repository-Level Coding	45
3.3	Positioning and Recovery Strategy Framework of the Present Study	56
4.1	Experimental Pipeline Overview	59
4.2	Repository Context Preparation	67
4.3	Patch Representation and Application	71
4.4	Information Available to Each Recovery Strategy	75
4.5	Outcome Classification State Machine	78
4.6	Patch Representation Before and After the Retrofit	80
5.1	Baseline Outcome Breakdown	87
5.2	Sub-Test Recovery on django__django-11019	90
5.3	Cross-Strategy Comparison Matrix	94
5.4	Recovery Cost by Strategy	95
6.1	The Recovery Strategy Ladder	111

LIST OF TABLES

Table No.	Title	Page Number
3.1	Literature Survey Table	47
3.2	Thematic Classification of Reviewed Literature	52
4.1	Task Selection — 20 Pilot Instances	63
4.2	Model and Access Configuration	64
4.3	Recovery Strategy Definitions	76
4.4	Outcome Categories	77
5.1	Baseline Results, All 20 Instances	85
5.2	Round One Recovery Outcomes	91
5.3	Round Two Recovery Outcomes	92
5.4	Coverage of the Logic-Class Comparison	92
5.5	Sub-Test Scores by Condition, Logic-Class Instances	93
5.6	Threats to Validity	102

CHAPTER 1

INTRODUCTION

1.1 Background

Recent advances in Large Language Models (LLMs) have substantially expanded the capabilities of artificial intelligence systems in software engineering. Modern LLMs can generate source code from natural-language descriptions, explain existing programs, identify defects, propose modifications, generate test cases, and assist developers across multiple stages of the software development lifecycle. As these capabilities have matured, research has progressed beyond systems that merely generate isolated code fragments toward autonomous coding agents capable of interacting with software repositories, using development tools, executing programs, observing outcomes, and iteratively modifying their own generated solutions [1]–[6].

An autonomous coding agent can be broadly understood as an LLM-driven system that combines language-based reasoning with external tools and an execution environment to perform software engineering tasks with limited direct human intervention. Unlike conventional code-generation systems, such agents may need to inspect a repository, identify relevant files, understand existing implementation dependencies, formulate a modification strategy, edit source code, execute tests, interpret execution results, and revise the proposed solution. Systems such as SWE-agent, AutoCodeRover, RepairAgent, and OpenHands demonstrate this transition toward repository-level and tool-using software engineering agents [3]–[6].

This transition is important because real-world software engineering problems rarely consist of generating a correct program from an empty context. Instead, developers typically work with existing repositories containing multiple source files, dependencies, configuration files, tests, historical design decisions, and implementation constraints. Consequently, solving a repository-level issue requires not only code generation but also contextual understanding, planning, repository navigation, tool use, debugging, and adaptation to feedback.

Despite the increasing autonomy of these systems, reliability remains a major challenge. An agent may generate a syntactically valid modification that nevertheless fails the intended tests. It may select an inappropriate implementation strategy, misunderstand the reported issue, modify the wrong section of a file, fail to account for interactions between components, or

produce an otherwise reasonable solution that cannot be correctly applied to the repository. These failure modes indicate that successful autonomous coding requires more than initial code-generation capability.

A particularly important aspect of reliability is the ability of an agent to recover after an unsuccessful attempt. In conventional software development, a failed test provides information that can be used to identify and correct the underlying defect. An autonomous coding agent should ideally exhibit a similar capability: detect the failure, determine its cause, revise its reasoning or implementation, apply an improved solution, and execute the relevant tests again.

However, failure detection and failure recovery are not equivalent. An agent may correctly observe that its previous solution failed while remaining unable to determine why it failed or how the solution should be changed. This distinction motivates the study of structured failure-recovery strategies involving retry, self-reflection, and evidence-based diagnosis.

The present dissertation investigates this problem empirically by comparing three recovery strategies for an LLM-based coding agent operating on repository-level software engineering tasks: **Blind Retry**, **Reflection-only**, and **Diagnose+Revise**. The study is grounded in the research gap identified through a systematic review of 35 studies published between 2023 and 2026 and evaluates the strategies using real repository-level tasks from the SWE-bench Lite benchmark.

1.2 Evolution of LLM-Based Coding Systems

The development of LLM-based software engineering systems can be viewed as a progression from code completion and generation toward increasingly autonomous, feedback-driven software engineering.

Early LLM-based coding applications primarily focused on generating code from natural-language prompts or completing partially written programs. Such systems were useful for reducing implementation effort but generally operated within a human-controlled workflow. The developer remained responsible for deciding which files to modify, executing the generated code, interpreting failures, and determining subsequent actions.

Subsequent research introduced iterative refinement mechanisms in which generated outputs could be evaluated and improved using additional feedback. Self-Refine demonstrated that an LLM can iteratively improve its own output using generated feedback [8], while

Reflexion introduced verbal reinforcement mechanisms in which agents record and use reflective information from previous attempts [7]. Self-debugging research further explored whether LLMs can identify and correct errors in generated programs [10].

The next stage involved integrating LLMs with external tools and environments. Tool-augmented systems can invoke APIs, execute code, interact with operating environments, and obtain observations that are unavailable from language reasoning alone [20], [24], [25]. This development enabled agents to operate in environments where actions have observable consequences.

In software engineering, these developments resulted in repository-level agents. SWE-agent demonstrated an agent-computer interface for performing automated software engineering tasks [4]. AutoCodeRover introduced autonomous program improvement through repository exploration and targeted code modification [3]. RepairAgent investigated autonomous LLM-based program repair [5], while OpenHands provided an open platform for generalist AI software development agents [6].

The resulting architecture differs fundamentally from one-shot code generation. A repository-level coding agent can be represented conceptually as an iterative process:

Task Description → Repository Understanding → Planning → Code Modification → Execution → Observation → Evaluation → Revision

The inclusion of an execution and evaluation stage is particularly important. The agent is no longer evaluated solely on whether generated code appears plausible; it can be evaluated according to whether the resulting repository satisfies executable tests or other task-specific criteria.

This shift also changes the nature of failure. In a one-shot generation setting, an incorrect output is simply an incorrect output. In an autonomous setting, failure becomes part of an interaction trajectory. The agent has an opportunity to observe the result of its action and potentially use that information to improve a subsequent attempt.

Therefore, the central research question is no longer only whether an LLM can generate a correct patch on its first attempt. It also becomes necessary to investigate whether the agent **can learn from an unsuccessful attempt within the same task trajectory and recover successfully.**

1.3 Autonomous Coding Agents

Autonomous coding agents combine several capabilities that traditionally belonged to separate software engineering tools. These capabilities commonly include natural-language reasoning, repository inspection, planning, tool invocation, code modification, execution, and feedback interpretation [1], [28]–[31].

A simplified representation of an autonomous coding-agent workflow is shown below:

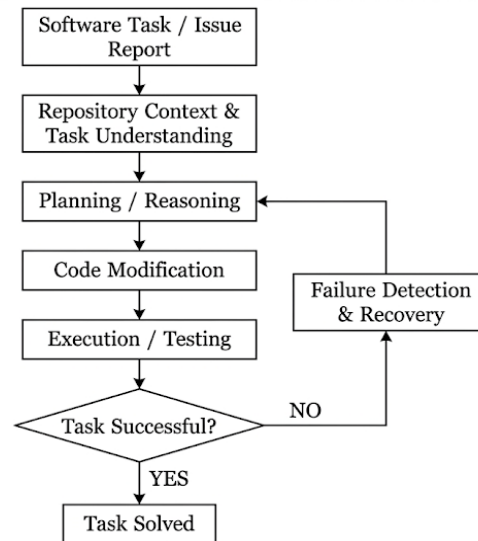


Figure 1.1 Autonomous Coding Agent Workflow

The recovery process is the primary focus of this dissertation.

A coding agent can fail at several distinct stages of this pipeline. Failure may occur because the agent misunderstands the issue description, selects an incorrect file, forms an incorrect hypothesis about program behavior, generates an incorrect modification, uses an inappropriate tool action, or produces a patch that cannot be applied correctly.

These failure types should not automatically be treated as equivalent. In particular, a failure in **reasoning about the program** differs from a failure in **representing or applying the generated patch**. The distinction became experimentally important in the present study because the initial evaluation infrastructure produced malformed patches under the unified-diff representation. This required a methodological correction before the recovery strategies could be compared fairly.

Accordingly, this dissertation treats coding-agent reliability as involving at least two related but distinguishable dimensions:

1. **Reasoning correctness** — whether the agent identifies and implements a technically correct solution to the software problem.

2. **Patch/application reliability** — whether the proposed modification can be represented, applied, and evaluated correctly within the target repository.

Separating these dimensions is necessary because an unsuccessful evaluation does not necessarily imply that the agent's underlying reasoning was incorrect. A patch may fail before its actual program-level correctness can be tested.

This distinction forms an important methodological component of the present research.

1.4 Failure in Repository-Level Coding

Repository-level software engineering tasks present substantially greater contextual requirements than isolated code-generation problems. A task may require understanding interactions between multiple modules, locating the relevant implementation, interpreting existing abstractions, preserving backward compatibility, and satisfying a pre-existing test suite.

A failure can therefore originate from several sources, including:

- incorrect interpretation of the issue description;
- incorrect localization of the relevant implementation;
- incomplete understanding of repository context;
- incorrect reasoning about program behavior;
- inappropriate modification of existing code;
- failure to consider edge cases;
- incorrect use of development tools;
- incorrect patch representation or application; and
- inability to interpret test or execution feedback.

From the perspective of failure recovery, the important distinction is between merely observing a failed outcome and using that outcome to produce a successful subsequent attempt.

For example, consider an agent that modifies a function and subsequently receives a failing test:

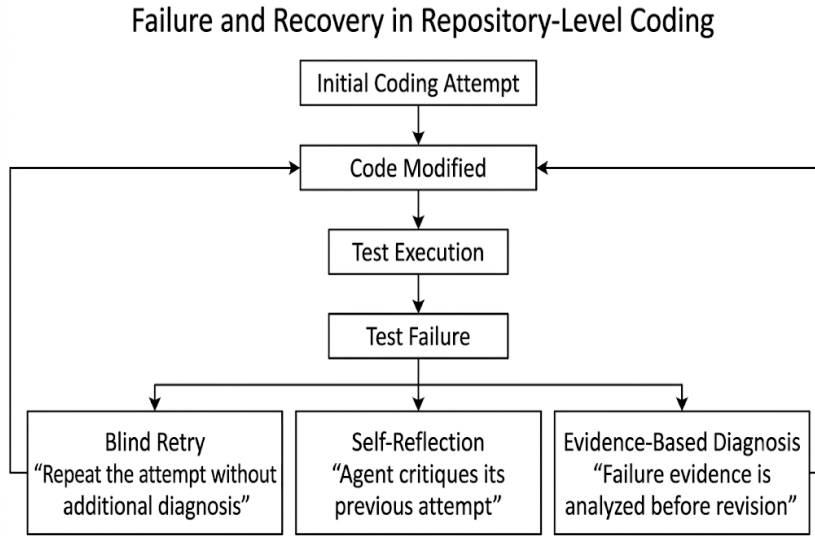


Figure 1.2 Failure and Recovery Strategies

Under a blind retry strategy, the agent receives no additional diagnostic information and repeats the task-solving process.

Under a reflection-based strategy, the agent is explicitly asked to reconsider its previous attempt and identify possible weaknesses in its reasoning.

Under a diagnosis-based strategy, the agent receives concrete evidence from execution and is asked to use that evidence to identify the underlying cause of failure before producing a revised solution.

These strategies differ not merely in prompt wording but in the **information available to the agent during recovery**. This provides a controlled basis for examining whether additional diagnostic information produces measurable improvement.

The present study therefore focuses specifically on the following empirical question:

Does providing an LLM-based coding agent with explicit, execution-grounded diagnostic evidence improve its ability to recover from a failed repository-level coding attempt compared with blind retry or self-reflection alone?

1.5 Self-Reflection and Failure Recovery

Self-reflection has emerged as an important mechanism for improving the behavior of LLM-based agents. Reflection-based methods encourage an agent to examine its previous reasoning, identify potential mistakes, and produce a revised solution. Reflexion, for example,

uses verbal feedback and reflective information to improve subsequent decisions [7]. Self-Refine similarly investigates iterative improvement through self-generated feedback [8]. Research on self-debugging examines whether language models can identify and correct errors in generated programs [10].

These approaches suggest that an agent's own previous attempt can serve as useful information for a subsequent attempt. However, self-reflection does not necessarily provide access to the actual cause of an execution failure.

This distinction can be represented as follows:

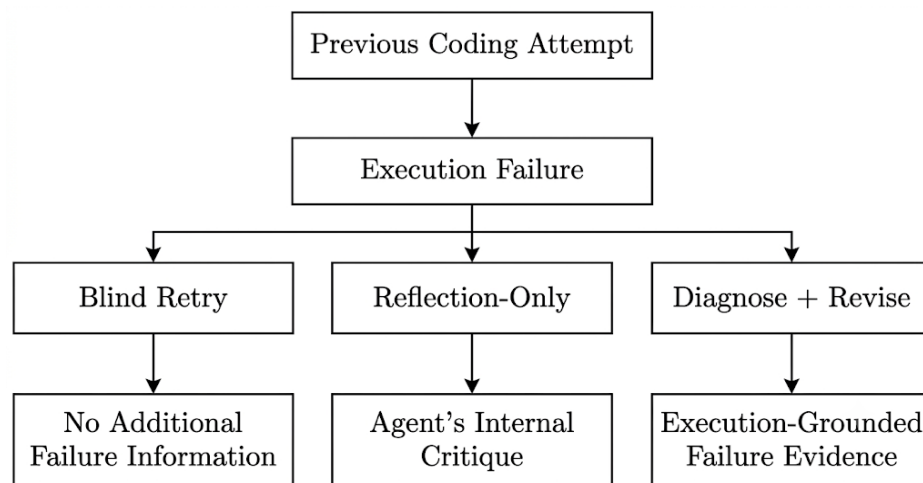


Figure 1.3 Information Available During Failure Recovery

The difference is particularly relevant for repository-level bugs because an unsuccessful patch may fail for reasons that are not immediately apparent from the generated code itself. The actual test output can provide information about the observed behavior of the program and may therefore provide a stronger basis for revision than introspection alone.

At the same time, the availability of diagnostic information does not guarantee that an LLM can use that information effectively. The agent must still connect the observed failure to the underlying program behavior, identify the appropriate correction, implement the correction, and preserve other required behavior.

Consequently, the effectiveness of diagnosis-based recovery is an empirical question rather than an assumption.

The systematic review conducted as the foundation of this dissertation examined research across self-reflection, memory, planning, tool use, failure analysis, and automated

repair. The review indicated substantial interest in these individual capabilities, but comparatively limited direct evaluation of whether structured, evidence-grounded recovery improves repository-level coding outcomes relative to simpler recovery baselines.

This observation provides the conceptual foundation for the experimental study presented in later chapters.

1.6 Motivation

The motivation for this research originates from the increasing autonomy of LLM-based software engineering systems and the corresponding need to understand their reliability after failure.

A system that succeeds only when its first generated solution is correct may remain unreliable for complex repository-level tasks. In contrast, an agent capable of recognizing and recovering from its own failures could potentially perform more robustly in realistic software engineering environments.

The first phase of this dissertation therefore consisted of a systematic review of 35 studies published between 2023 and 2026. The review investigated research concerning autonomous coding agents, self-reflection, memory, planning, tool use, failure recovery, automated debugging, and evaluation benchmarks. The review subsequently formed the basis of a manuscript submitted to *Information and Software Technology* under Manuscript No. INFSOF-D-26-01742.

The review motivated a more focused empirical investigation.

Rather than asking whether an LLM can solve repository-level tasks in general, this dissertation examines what happens **after the initial solution fails**.

Three increasingly informative recovery conditions are therefore compared:

1. Blind Retry

The agent is asked to attempt the task again without receiving additional information about the failure.

2. Reflection-only

The agent is shown its previous failed attempt and is explicitly instructed to reconsider and critique its earlier reasoning before generating a revised solution.

3. Diagnose+Revise

The agent is shown its previous attempt together with actual execution evidence describing what happened when that attempt was evaluated. It is then asked to diagnose the underlying problem and generate a revised solution.

The comparison allows the study to isolate an important question:

Is additional diagnostic information sufficient to improve recovery, or does successful recovery require deeper iterative reasoning, broader repository context, or multiple cycles of interaction?

The empirical evaluation also revealed an important methodological issue. Initial experiments using a unified-diff representation produced malformed patch failures that could obscure the actual reasoning capability of the agent. The evaluation pipeline was consequently redesigned using SEARCH/REPLACE edit blocks and programmatically constructed diffs. This allowed patch-formatting reliability to be separated from program-level reasoning correctness.

Thus, the motivation of the dissertation is both **substantive** and **methodological**:

- substantively, to investigate whether diagnosis-based recovery improves coding-agent performance after failure;
- methodologically, to establish an evaluation procedure that distinguishes reasoning failures from patch-generation and application failures.

1.7 Problem Statement

Although recent research has demonstrated substantial progress in autonomous coding agents, self-reflection, automated debugging, and LLM-based program repair, an important empirical question remains insufficiently resolved: **whether providing an autonomous coding agent with explicit, execution-grounded diagnosis of a previous failure actually improves its ability to recover from that failure on repository-level software engineering tasks.**

Existing approaches investigate several mechanisms for improving agent behavior, including self-reflection, experiential learning, memory, planning, tool use, and automated correction [7]–[17], [22]–[25]. However, these mechanisms do not necessarily establish that an agent can convert failure evidence into a correct repository-level repair.

Furthermore, evaluation of coding agents can conflate different failure sources. A task may be classified as unsuccessful because the generated patch is malformed or cannot be applied, even when the underlying reasoning may have been partially or fully correct. Conversely, a correctly applied patch may still fail because the agent's reasoning about the underlying software defect was incorrect.

The research problem addressed by this dissertation can therefore be stated as follows:

There is limited empirical evidence on whether execution-grounded, diagnosis-based recovery provides a measurable advantage over blind retry and self-reflection when an LLM-based coding agent attempts to recover from genuine repository-level logic failures, and existing evaluation procedures may conflate reasoning failures with patch-formatting failures.

This problem is investigated through a controlled comparison of Blind Retry, Reflection-only, and Diagnose+Revise strategies using repository-level tasks from SWE-bench Lite.

1.8 Research Questions

Based on the identified research gap and the experimental objectives, the dissertation investigates the following research questions.

RQ1. Recovery Effectiveness

Does diagnosis-based recovery improve the successful resolution of genuine repository-level coding failures compared with blind retry and reflection-only recovery?

This question evaluates whether explicit execution-grounded diagnostic information produces a measurable improvement in recovery success.

RQ2. Role of Self-Reflection

Does providing the agent with its own previous failed attempt and explicitly requesting reflection improve recovery compared with repeating the task without additional information?

This question isolates the potential contribution of self-reflection relative to blind retry.

RQ3. Failure-Type Separation

To what extent do patch-formatting and patch-application failures affect the evaluation of coding-agent recovery strategies, and can they be separated from genuine program-reasoning failures?

This question addresses the methodological issue discovered during the initial evaluation.

RQ4. Robustness of Recovery

Are the observed recovery outcomes consistent across independent recovery rounds when the agent is given another opportunity to revise its solution?

This question evaluates whether the primary comparison is stable across repeated recovery attempts rather than being dependent on a single revision round.

These questions collectively connect the literature-derived research gap to the experimental methodology presented in Chapter 4.

1.9 Research Contributions

The dissertation makes the following contributions.

1.9.1 Reproducible Failure-Recovery Evaluation Pipeline

A controlled experimental pipeline was developed for evaluating failure-recovery strategies in repository-level LLM coding agents. The pipeline integrates task selection, repository context preparation, agent prompting, patch generation, programmatic patch application, test execution, failure classification, and recovery evaluation.

1.9.2 Controlled Comparison of Recovery Strategies

The study provides an empirical comparison of three recovery conditions:

- Blind Retry;
- Reflection-only; and
- Diagnose+Revise.

The comparison is performed on a shared failure pool derived from an initial baseline evaluation, allowing the strategies to be examined under controlled conditions.

1.9.3 Separation of Patch Reliability and Reasoning Correctness

The study identifies and addresses a methodological confound between patch-formatting reliability and program-level reasoning correctness. The transition from the initial unified-diff representation to a SEARCH/REPLACE mechanism with programmatically constructed diffs enabled these two failure dimensions to be evaluated more independently.

1.9.4 Robustness-Checked Empirical Finding

The recovery comparison was conducted across two independent rounds. Following the fairness retrofit, the three recovery strategies produced identical outcomes on the genuine logic failures evaluated in the study. The Diagnose+Revise condition did not recover any of the observed genuine logic failures despite receiving fresh execution-grounded evidence in the second round.

This result is interpreted as a finding specific to the evaluated experimental setting rather than as evidence that diagnosis-based recovery is universally ineffective.

1.9.5 Methodological Evidence for Future Agent Evaluation

The study documents practical evaluation considerations relevant to repository-level coding-agent research, including patch representation, execution-environment consistency, failure classification, and the separation of infrastructure failures from reasoning failures.

1.10 Scope of the Dissertation

The scope of this dissertation is deliberately limited to failure recovery in LLM-based coding agents operating on repository-level software engineering tasks.

The empirical evaluation focuses on tasks drawn from the SWE-bench Lite benchmark. The study uses a controlled experimental setup in which the target file localization is obtained from gold-patch information. This oracle localization isolates the recovery and code-reasoning problem from the additional challenge of independently discovering the correct file within a repository.

The study evaluates three recovery strategies:

1. Blind Retry;
2. Reflection-only; and
3. Diagnose+Revise.

The core evaluation considers initial task success, genuine logic failure, patch-formatting/application failure, and subsequent recovery outcomes.

The study does not attempt to evaluate every capability of autonomous coding agents. In particular, it does not provide a comprehensive comparison of multiple LLM families, autonomous repository exploration strategies, long-term memory architectures, multi-agent collaboration frameworks, or unrestricted multi-round software development workflows.

Similarly, the findings should not be interpreted as a universal ranking of recovery mechanisms. The experimental results are based on a limited pilot failure pool and a specific model, environment, benchmark subset, and recovery protocol.

The limitations arising from these choices are discussed explicitly in Chapter 5 and revisited in Chapter 6.

1.11 Organization of the Dissertation

The remainder of this dissertation is organized as follows.

Chapter 2 — Aim, Objectives and Scope defines the overall aim of the research, presents the specific research objectives, formulates the experimental direction, and establishes the scope and assumptions of the study.

Chapter 3 — Literature Review presents the systematic literature foundation of the dissertation. It examines research on autonomous coding agents, self-reflection, memory, planning, tool use, failure detection, automated debugging, program repair, and evaluation benchmarks. The chapter synthesizes 35 studies published between 2023 and 2026 and identifies the research gap that motivates the empirical investigation.

Chapter 4 — Methodology describes the experimental design in detail. It presents the SWE-bench Lite task selection, repository context, model and access configuration, oracle file localization, patch representation, evaluation infrastructure, baseline procedure, three recovery conditions, fairness retrofit, evaluation metrics, and second recovery round.

Chapter 5 — Results and Discussion presents the baseline observations and comparative results of the three recovery strategies. It examines the initial patch-formatting problem, the corrected evaluation, the results of the first recovery round, the robustness check using the second round, and the implications of the observed null result. Threats to validity and experimental limitations are also discussed.

Chapter 6 — Conclusions and Future Work summarizes the research, presents the principal findings and contributions, discusses the limitations of the current evaluation, and identifies directions for future research, including larger-scale evaluation, multi-round recovery, richer repository context, multiple LLMs, and recovery-oriented evaluation metrics.

The dissertation concludes with the **References** and supporting material in the **Appendices**, including experimental prompts, task-level results, implementation details, and reproducibility information where appropriate.

CHAPTER 2

AIM, OBJECTIVES AND RESEARCH SCOPE

2.1 Aim of the Research

The primary aim of this research is to investigate whether self-reflection and failure-recovery mechanisms can improve the ability of repository-level agentic AI coding systems to recover from failed coding attempts.

Large language model (LLM)-based coding systems have increasingly progressed from generating isolated code fragments to performing multi-step software engineering tasks involving repository understanding, code modification, execution, testing, and iterative revision. However, successful completion of a repository-level software engineering task depends not only on the generation of an initial code modification but also on the ability of the coding system to respond appropriately when that modification fails.

This research therefore focuses specifically on the failure-recovery stage of an agentic coding workflow. The study examines three recovery strategies: Blind Retry, Reflection-only, and Diagnose+Revise. The investigation considers whether providing an agent with additional reasoning or execution-grounded failure information leads to improved recovery from unsuccessful coding attempts.

The research is grounded in two complementary components. First, a systematic review of 35 studies published between 2023 and 2026 was conducted to examine existing work related to autonomous coding agents, self-reflection, memory, planning, failure detection, failure recovery, automated debugging, and evaluation of agentic coding systems. Second, an empirical evaluation is conducted using repository-level software engineering tasks from SWE-bench Lite to examine the behaviour of different failure-recovery strategies under controlled experimental conditions.

The overall aim is therefore not merely to determine whether an agent can generate a correct patch, but to study the process that occurs after an initial coding attempt fails and to distinguish failures caused by patch representation or application mechanisms from failures that require genuine reasoning about the software task.

2.2 Research Objectives

The objectives of the present study are formulated based on the research gap identified through the systematic review and the requirements of the empirical investigation.

Objective 1: To systematically examine existing research on failure recovery in agentic coding systems

To review and synthesize relevant research published between 2023 and 2026 concerning autonomous coding agents, self-reflection, memory mechanisms, planning, tool use, failure detection, automated debugging, program repair, and evaluation of repository-level coding systems.

Objective 2: To identify the role of self-reflection in agentic coding workflows

To examine how self-reflection and iterative reasoning are incorporated into existing LLM-based coding systems and how these mechanisms are intended to support correction following unsuccessful coding attempts.

Objective 3: To identify and characterize failure-recovery mechanisms

To examine the mechanisms used by agentic coding systems to detect, analyze, and respond to failed coding attempts, with particular attention to the distinction between simple retry behaviour, reflection-based revision, and diagnosis-guided revision.

Objective 4: To design a controlled experimental framework for evaluating recovery strategies

To develop an experimental framework for comparing different failure-recovery strategies within a repository-level coding environment while controlling factors that may independently influence the observed outcome.

Objective 5: To compare Blind Retry, Reflection-only, and Diagnose+Revise strategies

To empirically compare the behaviour of three recovery strategies:

1. **Blind Retry**, in which the agent attempts another solution without an additional structured diagnosis;
2. **Reflection-only**, in which the agent uses its own critique of the previous attempt to guide another attempt; and
3. **Diagnose+Revise**, in which execution-grounded failure information is incorporated into the diagnosis and subsequent revision process.

Objective 6: To distinguish genuine reasoning failures from patch-formatting or patch-application failures

To identify whether an observed failure is attributable to the correctness of the agent's software reasoning or to the technical mechanism used to represent and apply the generated patch.

This objective is important because a failure to apply a patch correctly does not necessarily demonstrate that the agent was unable to reason about the underlying software problem. Therefore, the experimental methodology incorporates mechanisms intended to reduce patch-formatting and patch-application effects when evaluating recovery behaviour.

Objective 7: To evaluate the robustness of recovery behaviour across repeated evaluation rounds

To examine whether the observed behaviour of the recovery strategies remains consistent when the evaluation is repeated under the controlled experimental setup.

Objective 8: To derive methodological implications for the evaluation of self-reflection in coding agents

To identify methodological considerations that should be taken into account when evaluating self-reflection and failure-recovery mechanisms, particularly the need to separate failures arising from infrastructure or patch representation from failures requiring genuine task-level reasoning.

2.3 Research Questions

The research questions are derived from the research gap identified in the systematic review and the objectives of the empirical investigation.

RQ1. What failure-recovery mechanisms have been investigated in existing research on agentic AI coding systems?

This question addresses the systematic-review component of the study. It examines how existing studies approach failure detection, self-reflection, diagnosis, memory, replanning, iterative execution, and recovery in autonomous coding systems.

RQ2. How do Blind Retry, Reflection-only, and Diagnose+Revise differ in their approach to recovering from failed repository-level coding attempts?

This question focuses on the methodological and behavioural differences between the three recovery strategies investigated in the empirical study.

RQ3. Does the availability of reflection or execution-grounded diagnostic information improve recovery from failed coding attempts?

This question examines whether additional information available after a failure results in improved recovery compared with repeating a coding attempt without additional structured information.

RQ4. To what extent can observed recovery failures be attributed to reasoning limitations rather than patch-formatting or patch-application mechanisms?

This question addresses a central methodological issue identified during the experimental investigation. It considers whether an unsuccessful recovery attempt reflects a failure of the agent's reasoning about the software task or a failure in the technical representation and application of the proposed code modification.

RQ5. Are the observed recovery behaviours consistent across repeated evaluation rounds under controlled experimental conditions?

This question examines the robustness and repeatability of the observed behaviour of the evaluated recovery strategies.

2.4 Scope of the Study

The scope of the research is defined to maintain a clear distinction between the broad field of agentic software engineering and the specific failure-recovery problem investigated in this dissertation.

2.4.1 Scope of the Systematic Review

The systematic review considers 35 studies published between 2023 and 2026. The review focuses on research related to LLM-based autonomous coding systems and concepts relevant to their ability to perform software engineering tasks.

The review particularly considers the following areas:

- autonomous and agentic coding systems;
- self-reflection and iterative reasoning;
- memory mechanisms;
- planning and tool use;
- failure detection and failure analysis;
- automated debugging and program repair;

- repository-level software engineering;
- evaluation benchmarks and evaluation methodologies.

The purpose of the review is to establish the current research landscape and identify limitations or gaps relevant to failure recovery in repository-level agentic coding systems.

2.4.2 Scope of the Empirical Study

The empirical component focuses on repository-level software engineering tasks evaluated using SWE-bench Lite.

The investigation is specifically concerned with the behaviour of a coding agent after an initial coding attempt fails. The study therefore does not attempt to provide a general evaluation of all capabilities of LLM-based coding agents.

The experimental comparison is limited to three recovery strategies:

- Blind Retry;
- Reflection-only; and
- Diagnose+Revise.

The investigation considers the ability of these strategies to respond to failed coding attempts and examines the effect of providing different forms of information during recovery.

2.4.3 Scope of Failure Analysis

The study distinguishes between different sources of unsuccessful outcomes. In particular, it considers the difference between:

- failures associated with patch representation or application; and
- failures associated with the correctness of the underlying software reasoning.

This distinction is necessary because an experimental system may fail to apply a technically valid reasoning process if the generated modification is represented in an unsuitable patch format.

2.4.4 Scope of Evaluation

The evaluation focuses on task completion and recovery behaviour under the defined experimental conditions. The study does not attempt to evaluate every dimension of agentic coding quality, such as long-term software maintenance, production deployment, team collaboration, user experience, or organizational impact.

The conclusions are therefore limited to the experimental setting, task benchmark, recovery strategies, and evaluation procedure used in this dissertation.

2.5 Research Assumptions

The following assumptions are adopted to establish a consistent basis for the experimental investigation.

Assumption 1: Repository-level coding tasks provide a meaningful setting for studying failure recovery

Repository-level software engineering tasks require an agent to understand existing code, identify relevant files or program components, modify code, and verify the resulting behaviour. Therefore, they provide an appropriate environment for investigating failure and recovery behaviour.

Assumption 2: Test execution provides useful evidence about coding outcomes

The study assumes that the execution of the relevant tests provides observable evidence about whether a generated modification satisfies the expected behaviour of the task.

Assumption 3: Recovery strategies can be compared under controlled conditions

The experimental design assumes that the recovery strategies can be evaluated while keeping the underlying task environment and major experimental conditions sufficiently controlled to permit comparison.

Assumption 4: Patch-application reliability is a separate methodological factor

The study assumes that the mechanism used to represent and apply a generated modification can affect the observed experimental outcome independently of the agent's reasoning about the task. Consequently, patch-formatting and application issues should be considered separately from genuine reasoning failures.

Assumption 5: Repeated evaluation can provide evidence about consistency

The study assumes that repeating the evaluation under the defined experimental conditions can provide useful evidence about whether an observed behaviour is persistent or dependent on a particular evaluation round.

2.6 Limitations of the Study

The present research has several limitations that should be considered when interpreting its findings.

2.6.1 Limited Benchmark Scope

The empirical investigation is conducted using SWE-bench Lite rather than the complete range of repository-level software engineering tasks. Consequently, the findings should not automatically be generalized to all software repositories, programming languages, development environments, or coding-agent applications.

2.6.2 Limited Number of Evaluated Tasks

The empirical evaluation is based on a defined experimental task set rather than an exhaustive sample of repository-level software engineering problems. The size and composition of the evaluated task set therefore limit the extent to which the findings can be generalized.

2.6.3 Dependence on the Experimental Model and Infrastructure

The observed behaviour of an agentic coding system can depend on the underlying language model, prompting configuration, repository context, execution environment, tool configuration, and other infrastructure choices. Therefore, the findings describe the evaluated experimental configuration rather than all possible LLM-based coding agents.

2.6.4 Limited Recovery Strategies

The study evaluates three specific recovery strategies: Blind Retry, Reflection-only, and Diagnose+Revise. Other recovery mechanisms, including more sophisticated memory, planning, multi-agent collaboration, retrieval-based approaches, or alternative diagnosis mechanisms, are outside the primary scope of the empirical comparison.

2.6.5 Difficulty of Isolating Reasoning Failures

Determining whether an unsuccessful coding attempt represents a reasoning failure can be challenging because multiple components of an agentic coding pipeline interact. The study therefore attempts to control patch-formatting and patch-application effects, but complete isolation of all possible sources of failure may not be possible.

2.6.6 Limited Generalization of Null or Negative Findings

If a recovery strategy does not improve task outcomes within the evaluated experimental setting, such a finding should not be interpreted as evidence that the underlying concept of self-

reflection or diagnosis is universally ineffective. It indicates only that an improvement was not demonstrated under the specific conditions evaluated in this study.

2.6.7 Dependence on Available Execution Evidence

The Diagnose+Revise strategy relies on information obtained from the execution and evaluation process. The quality and usefulness of this information may vary depending on the task, available tests, failure messages, and execution environment.

2.7 Chapter Summary

This chapter defined the aim, objectives, research questions, scope, assumptions, and limitations of the present dissertation. The research aims to investigate failure recovery in repository-level agentic AI coding systems, with particular emphasis on the role of self-reflection and execution-grounded diagnosis following unsuccessful coding attempts.

The study combines a systematic review of 35 studies published between 2023 and 2026 with an empirical investigation using repository-level tasks from SWE-bench Lite. The empirical component compares Blind Retry, Reflection-only, and Diagnose+Revise strategies and explicitly considers the distinction between patch-formatting or patch-application failures and failures associated with the underlying reasoning required to solve a software engineering task.

The research questions establish the basis for examining existing failure-recovery mechanisms, comparing recovery strategies, evaluating the value of additional failure information, distinguishing technical patch failures from reasoning failures, and examining the consistency of observed behaviour across evaluation rounds. The defined scope and limitations establish the boundaries within which the findings of the dissertation should be interpreted.

CHAPTER 3

LITERATURE REVIEW

3.1 Overview

The rapid development of Large Language Models (LLMs) has significantly changed the way artificial intelligence can be applied to software engineering. Earlier code-generation systems were primarily designed to generate short code fragments, complete functions, or provide programming assistance. Recent LLM-based systems have progressed toward autonomous or agentic coding systems capable of interacting with software repositories, examining existing source code, modifying files, executing tools, running tests, and revising their outputs based on observed results.

The development of agentic coding systems has introduced a shift from **single-step code generation** toward **multi-step software engineering workflows**. In such workflows, an agent may receive a natural-language issue description, inspect a repository, identify relevant files, formulate a solution, modify source code, execute tests, observe the results, and attempt further modifications when the initial solution is unsuccessful. Systems such as SWE-agent, AutoCodeRover, RepairAgent, and OpenHands demonstrate this transition toward autonomous software engineering.

However, increased autonomy also introduces new failure modes. An agent may misunderstand the task, select an incorrect file, generate an incorrect modification, misuse a development tool, fail to interpret test output, or produce a technically unsuitable patch. Therefore, the ability of an agent to **recover from a failed attempt** is an important component of reliable autonomous software engineering.

The literature on LLM agents has investigated several mechanisms intended to improve performance after an unsuccessful attempt. These include self-reflection, self-feedback, self-debugging, memory, planning, tool use, experience accumulation, automated diagnosis, and iterative repair. Reflexion introduced verbal reinforcement through feedback from previous attempts, while Self-Refine demonstrated iterative generation, feedback, and refinement using a single language model.

At the same time, research has increasingly focused on evaluating coding agents using realistic software engineering benchmarks. SWE-bench, for example, contains real GitHub

issues and corresponding software repositories, requiring systems to make repository-level changes rather than simply generate isolated functions.

The present chapter reviews the relevant literature across autonomous coding agents, self-reflection, memory, planning, failure analysis, automated debugging, program repair, and evaluation benchmarks. The purpose is not only to summarize previous research but also to identify the specific research gap addressed by this dissertation.

3.2 Autonomous Coding Agents

Autonomous coding agents represent a progression from conventional code-completion systems toward systems capable of performing multiple software engineering actions autonomously. Instead of producing a single code fragment in response to a prompt, an agentic coding system can interact with a repository and development environment through a sequence of actions.

A general autonomous coding workflow can be represented as:

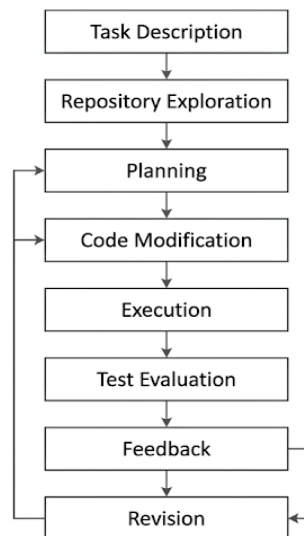


Figure 3.1 General Workflow of an Autonomous Coding Agent

This workflow requires the system to maintain an understanding of the task while interacting with an external software environment.

Surveys of LLM-based autonomous agents generally describe an agent as a system combining an LLM with additional components such as planning, memory, tool use,

perception, and action. Wang et al. presented a broad survey of LLM-based autonomous agents and proposed a unified view of agent construction and applications.

The transition to coding agents is particularly important because software engineering tasks require interaction with structured environments. A coding agent must understand repository organization, identify relevant source files, interpret dependencies, make modifications, and verify whether those modifications satisfy the requested behaviour.

3.2.1 SWE-agent

SWE-agent introduced an agent-computer interface specifically designed to support language-model agents in software engineering environments. Rather than treating the language model as a simple code generator, the system provides interfaces for navigating repositories, editing files, executing programs, and running tests. The study demonstrated that interface design itself can significantly affect agent performance.

This work is important to the present dissertation because it demonstrates that successful repository-level coding depends not only on model capability but also on the tools and interaction mechanisms provided to the model.

3.2.2 AutoCodeRover

AutoCodeRover investigates autonomous program improvement using an LLM-based system that combines repository exploration and code modification. The system focuses on identifying relevant program locations and generating modifications for real software issues. The work demonstrates the feasibility of autonomous program improvement at repository level.

However, successful patch generation does not necessarily imply that an agent can recover after a failed patch. This distinction is important for the present research.

3.2.3 RepairAgent

RepairAgent treats program repair as an autonomous agent task. The system can gather information about a bug, collect repair ingredients, invoke tools, and validate candidate fixes. Unlike fixed one-shot repair approaches, RepairAgent dynamically interacts with the development environment and uses feedback from previous attempts.

The work demonstrates the value of combining LLM reasoning with tool interaction and iterative validation. However, its primary focus is autonomous program repair rather than a controlled comparison between different post-failure recovery strategies.

3.2.4 OpenHands

OpenHands provides an open platform for AI software developers as generalist agents. Such systems aim to allow LLMs to interact with development environments and perform broader software engineering activities.

The importance of these systems is that they demonstrate a transition from **LLM as coding assistant** toward **LLM as autonomous software engineering actor**.

Nevertheless, the increased complexity of an agentic workflow also makes it more difficult to determine why a particular task fails. A failure may originate from planning, repository understanding, tool interaction, code generation, patch application, or reasoning.

3.3 Self-Reflection and Iterative Reasoning

Self-reflection refers to mechanisms through which an LLM examines its previous output, identifies possible weaknesses, and generates a revised solution. The underlying idea is that a model may improve its performance by using information from its own previous attempt rather than generating every solution independently.

This concept has become an important component of LLM-agent research.

3.3.1 Reflexion

Shinn et al. introduced Reflexion, a framework in which language agents use verbal feedback from previous attempts as a form of reinforcement. Instead of modifying model parameters, the system stores textual reflections that can influence subsequent attempts.

The important concept introduced by Reflexion is that **experience from a failed attempt can be converted into textual information and reused during future decision-making**.

For software engineering, this suggests a possible workflow:

Initial Attempt → Failure → Reflection → Revised Attempt

However, a reflection mechanism does not automatically guarantee that the model identifies the true cause of failure. A model may generate a plausible explanation that is unrelated to the actual software defect.

This distinction becomes particularly important in the present dissertation, where Reflection-only is evaluated separately from a condition receiving actual execution-grounded failure information.

3.3.2 Self-Refine

Madaan et al. introduced Self-Refine, in which an LLM generates an initial output, provides feedback on that output, and then uses the feedback to generate a refined output. The method does not require additional model training and uses the same LLM for generation, feedback, and refinement.

Self-Refine demonstrates that iterative feedback can improve performance across multiple tasks. However, the feedback mechanism is primarily based on model-generated feedback rather than necessarily being grounded in an external execution environment.

For coding agents, this raises an important question:

Does self-generated reflection provide enough information to recover from a genuine repository-level coding failure?

The present dissertation investigates this question through the Reflection-only condition.

3.3.3 Self-Debugging

Chen et al. proposed a self-debugging approach in which LLMs are encouraged to inspect and explain their generated programs and use execution-related information to identify mistakes. The work demonstrates that LLMs can perform certain forms of debugging without direct human intervention.

This research is closely related to the present study because it connects code generation with debugging rather than treating generation as a single isolated step.

However, self-debugging approaches may vary significantly in the amount and type of feedback provided to the model. Therefore, controlled experiments are required to determine whether specific forms of feedback actually improve recovery.

3.4 Memory Mechanisms in LLM Agents

Memory is another important component of agentic systems. A coding agent may encounter similar errors across different tasks, and retaining information about previous failures could potentially allow the agent to improve over time.

Memory mechanisms can broadly be divided into:

1. **Short-term or contextual memory**
2. **Long-term memory**
3. **Experience-based memory**
4. **Structured or agentic memory**

3.4.1 Experiential Learning

ExpeL investigates how LLM agents can accumulate experiences and extract useful knowledge from previous tasks. The agent recalls extracted experiences during future decision-making rather than requiring model parameter updates. The study reports that performance can improve as relevant experiences accumulate.

This concept is relevant to failure recovery because a failed coding attempt can potentially become an experience that informs future attempts.

3.4.2 Long-Term Memory

Maharana et al. investigated long-term memory in LLM agents using the LoCoMo benchmark. Their results demonstrate that LLMs face difficulties in maintaining long-range temporal and causal information across extended interactions, although long-context approaches and retrieval can provide improvements.

This suggests that simply providing an agent with more historical information does not guarantee effective use of that information.

3.4.3 A-MEM

A-MEM introduces an agentic memory system that dynamically organizes memories into interconnected knowledge structures. The system can create new memory entries, identify relationships between memories, and update existing representations as new information becomes available.

The relevance to failure recovery is significant: a future coding agent could potentially retain information about previously encountered failure types, diagnoses, and successful recovery actions.

However, the present dissertation does not implement persistent memory. Instead, it isolates the immediate recovery stage to determine whether information provided after a failure is sufficient to produce a corrected solution.

3.5 Planning and Tool Use

Autonomous coding requires more than code generation. The agent must decide **what action to perform next**.

Typical actions may include:

- inspecting files;
- searching for symbols;
- reading documentation;
- executing commands;
- running tests;
- modifying source code;
- inspecting errors;
- revising the proposed solution.

Masterman et al. reviewed emerging agent architectures involving reasoning, planning, and tool calling and highlighted the importance of these components in complex agent systems.

Planning can therefore be viewed as a mechanism connecting the agent's reasoning to its available tools.

However, planning itself can fail. An agent may choose an inappropriate action, make an incorrect assumption, or continue following a flawed plan even after receiving evidence that the plan has failed.

This creates a connection between **planning failure** and **failure recovery**.

A robust coding agent should therefore be capable of:

Plan → Execute → Observe → Diagnose → Replan → Execute Again
rather than simply:

Plan → Execute → Retry

3.6 Failure Detection and Failure Analysis

As agentic systems become more complex, understanding why an agent fails becomes increasingly important.

A task failure does not necessarily have a single cause. Potential failure categories include:

- task-understanding failure;
- repository-navigation failure;
- file-localization failure;
- planning failure;
- tool-use failure;
- code-generation failure;
- test-interpretation failure;
- patch-formatting failure;
- patch-application failure;
- reasoning failure;
- execution-environment failure.

Recent research has started to explicitly study failure taxonomies for LLM agents. Zhu et al. introduced AgentErrorTaxonomy and AgentDebug to classify failure modes across memory, reflection, planning, action, and system-level operations. Their results suggest that targeted debugging feedback can improve recovery in several general agent environments.

However, these experiments primarily concern environments such as ALFWorld, GAIA, and WebShop rather than repository-level software engineering tasks. This difference is important because software repositories introduce additional technical failure modes, particularly code modification and patch application.

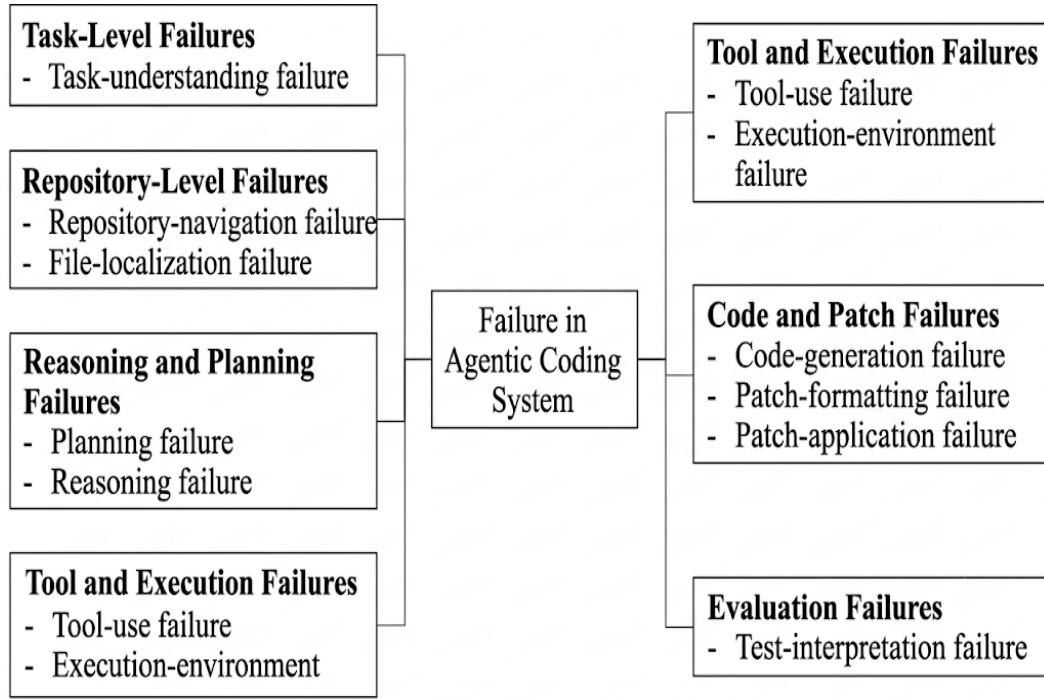


Figure 3.2 Failure Categories in Repository-Level Coding

3.7 Automated Debugging and Program Repair

Automated program repair (APR) has traditionally focused on automatically generating patches that correct software defects.

LLMs have expanded this field by enabling natural-language reasoning over source code and bug descriptions.

Zhang et al.'s systematic literature review of LLM-based automated program repair reflects the rapid growth of this research area and the increasing diversity of techniques used to generate and validate repairs.

RepairAgent represents an important transition toward autonomous program-repair agents. Rather than producing a patch in a fixed pipeline, the agent dynamically gathers information, uses tools, generates candidate fixes, and validates them.

The literature therefore demonstrates substantial progress in **generating and validating repairs**.

However, a distinct question remains:

When the first repair attempt fails, what information and recovery mechanism actually enables the next attempt to succeed?

This question is central to the present dissertation.

3.8 Evaluation Benchmarks for Coding Agents

Evaluation benchmarks are essential for determining whether improvements in agentic coding systems are genuine.

3.8.1 SWE-bench

SWE-bench introduced a repository-level benchmark containing 2,294 real-world software engineering problems collected from GitHub issues and corresponding pull requests across 12 Python repositories. Tasks require the system to modify a codebase to resolve a reported issue.

Unlike traditional programming benchmarks, SWE-bench frequently requires understanding interactions between multiple functions, classes, and files. It therefore provides a more realistic environment for studying autonomous software engineering.

The present dissertation uses **SWE-bench Lite** as the empirical task environment.

3.8.2 AgentBench

AgentBench provides a multidimensional evaluation framework covering eight interactive environments. It evaluates LLMs as agents rather than only measuring isolated generation quality.

AgentBench demonstrates the importance of evaluating agents through interactive tasks, but it is not specifically designed around repository-level software repair.

3.8.3 RedCode

RedCode focuses on safety-related evaluation of code agents. It contains risky code execution and code-generation tasks designed to examine whether agents recognize and handle potentially harmful operations.

The benchmark illustrates that evaluation of coding agents extends beyond functional correctness to include safety and responsible execution.

3.8.4 Evaluation Beyond Task Success

A major methodological issue is that a simple PASS/FAIL result may hide the actual reason for failure.

For example:

Agent reasoning correct → patch malformed → FAIL

and

Agent reasoning incorrect → patch applied successfully → FAIL

would appear identical in a conventional evaluation.

The present research therefore explicitly distinguishes **patch-formatting/application failures** from **genuine reasoning failures**.

3.9 Comparative Analysis of the Reviewed Studies

The following table presents the principal literature reviewed for this dissertation. The table follows the format requested by the supervisor and extends it with a research-gap/relevance column because identifying the gap is central to this dissertation.

Table 3.1: Literature Survey Table

Sr No.	Paper Name	Year	Methods	Advantages	Disadvantages/ Research Gap
1	LLM-based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead [1]	2025	Reviews LLM-based multi-agent systems and their applications in software engineering.	Provides a broad research landscape and identifies future directions.	Broad review; does not experimentally isolate post-failure recovery strategies.
2	The Rise of AI Teammates in Software Engineering (SE 3.0) [2]	2025	Conceptual analysis of autonomous AI teammates and their role in software engineering.	Provides a broader view of autonomous software development.	Primarily conceptual; limited controlled empirical analysis of failure recovery.
3	AutoCodeRover: Autonomous Program Improvement [3]	2024	Uses LLM-driven repository exploration and program modification for issue resolution.	Demonstrates autonomous repository-level program improvement.	Focuses on program improvement rather than controlled comparison of recovery mechanisms.
4	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering [4]	2024	Provides an agent-computer interface for repository navigation, editing, execution, and testing.	Demonstrates importance of specialized interfaces for coding agents.	Does not isolate the contribution of reflection versus diagnosis after failure.
5	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair [5]	2025	Autonomous planning, information gathering, tool use, repair generation, and validation.	Integrates reasoning, tools, and iterative validation.	Repair performance is not structured as a controlled comparison of blind retry, reflection, and diagnosis.
6	OpenHands: An Open Platform for AI Software Developers as Generalist Agents [6]	2025	Generalist coding agent capable of interacting with software-development environments.	Broad autonomous software-development capabilities.	General-purpose architecture makes it difficult to isolate individual recovery mechanisms.
7	Reflexion: Language Agents with Verbal Reinforcement Learning [7]	2023	Stores verbal reflections from previous attempts and uses them in subsequent attempts.	Text-based reflection enables immediate, update-free learning.	Reflection may not identify the actual root cause of a coding failure.
8	Self-Refine: Iterative Refinement with Self-Feedback [8]	2023	Generates output, critiques it, and iteratively refines the output.	Simple and training-free iterative improvement framework.	Feedback is generated by the model itself and may be inaccurate.
9	Self-Reflection in LLM Agents: Effects on Problem-Solving Performance [9]	2024	Studies the effect of self-reflection on agent problem-solving.	Directly investigates reflection as an agent capability.	General problem-solving setting; limited repository-level software engineering focus.
10	Teaching Large Language Models to Self-Debug [10]	2024	Uses code explanation and execution to enable models to identify and correct errors.	Demonstrates LLM-based debugging without requiring direct human feedback.	Does not specifically study controlled recovery after repository-level failure.
11	ExpeL: LLM Agents Are Experiential Learners [11]	2024	Extracts experiences and knowledge from previous tasks for later reuse.	Demonstrates learning from accumulated experience without parameter updates.	Focuses on experiential learning rather than immediate diagnosis of a failed coding patch.
12	Reflective Multi-Agent Collaboration Based on Large Language Models [12]	2024	Uses reflection within multi-agent collaboration.	Combines collaboration and reflection.	Multi-agent architecture introduces additional variables and does not

Sr No.	Paper Name	Year	Methods	Advantages	Disadvantages/ Research Gap
					isolate single-agent recovery.
13	Automatically Correcting Large Language Models: Surveying the Landscape of Diverse Automated Correction Strategies [13]	2024	Surveys training-time, generation-time, and post-hoc correction using automated feedback.	Provides broad taxonomy of automated correction mechanisms.	Broad LLM scope; not specifically focused on repository-level coding recovery.
14	OS-Copilot: Towards Generalist Computer Agents with Self-Improvement [14]	2024	Develops generalist computer agents with self-improvement mechanisms.	Demonstrates autonomous computer interaction and improvement.	General computer-agent setting rather than controlled coding-failure recovery.
15	A-MEM: Agentic Memory for LLM Agents [15]	2025	Dynamically organizes and links agent memories.	Provides adaptive memory organization and experience retrieval.	Memory mechanism is not directly evaluated for repository-level coding failure recovery.
16	MIRIX: Multi-Agent Memory System for LLM-Based Agents [16]	2025	Uses multi-agent memory architecture for storing and retrieving information.	Explores structured memory for long-term agent behaviour.	Additional memory complexity; limited direct evidence for coding-failure recovery.
17	Evaluating Very Long-Term Conversational Memory of LLM Agents [17]	2024	Introduces long-term memory benchmark and evaluates extended conversations.	Provides systematic evaluation of long-term memory.	Conversational setting differs from repository-level software engineering.
18	A Systematic Literature Review on Large Language Models for Automated Program Repair [18]	2026	Systematically reviews LLM-based automated program repair techniques.	Provides comprehensive view of LLM-based APR research.	Repair approaches are well-studied, but post-failure recovery effectiveness remains underexplored.
19	SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [19]	2024	Evaluates LMs on 2,294 real GitHub software-engineering problems.	Provides realistic repository-level benchmark.	Original benchmark measures issue resolution rather than isolating recovery mechanisms.
20	AgentBench: Evaluating LLMs as Agents [20]	2024	Multi-dimensional evaluation across eight interactive environments.	Evaluates agents in interactive rather than isolated generation settings.	Not specifically designed for coding-agent failure recovery.
21	Where LLM Agents Fail and How They Can Learn From Failures [21]	2025	Introduces failure taxonomy, AgentErrorBench, and AgentDebug for failure diagnosis and recovery.	Directly connects failure analysis with corrective feedback and recovery.	Evaluated primarily in general agent environments rather than repository-level coding tasks.
22	TPTU: Task Planning and Tool Usage of Large Language Model-Based AI Agents [22]	2023	Studies planning and tool-use behaviour in LLM agents.	Highlights planning and tool use as central agent capabilities.	Does not focus specifically on coding failure recovery.
23	LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with LLMs [23]	2023	Uses LLMs for grounded planning and action selection.	Demonstrates LLM-based planning for complex tasks.	Embodied-agent environment is substantially different from repository-level software engineering.
24	API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs [24]	2023	Evaluates LLMs in API/tool-use scenarios.	Provides systematic evaluation of tool-augmented behaviour.	Tool-use benchmark rather than repository-level coding recovery.

Sr No.	Paper Name	Year	Methods	Advantages	Disadvantages/ Research Gap
25	LLM with Tools: A Survey [25]	2024	Surveys tool-augmented LLM systems.	Provides overview of tool integration and agent capabilities.	Broad survey; does not experimentally isolate recovery strategies.
26	Agent-as-a-Judge: Evaluate Agents with Agents [26]	2024	Uses agents to evaluate other agents and their outputs.	Explores scalable agent evaluation.	Evaluation methodology rather than failure recovery mechanism.
27	RedCode: Risky Code Execution and Generation Benchmark for Code Agents [27]	2024	Evaluates risky code execution and generation using dedicated benchmark tasks.	Highlights safety-related failure modes in coding agents.	Focuses on safety rather than functional recovery from software bugs.
28	The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling [28]	2024	Surveys single-agent and multi-agent architectures, planning, reasoning, and tool execution.	Provides architectural perspective on agent design.	Does not experimentally test repository-level recovery strategies.
29	The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges [29]	2025	Reviews definitions, architectures, applications, metrics, and challenges of agentic AI.	Provides broad conceptual framework for agentic AI.	General agentic AI focus; coding-specific failure recovery is not the central subject.
30	Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions [30]	2026	Comprehensive survey of agentic AI architectures and applications.	Provides current overview of the rapidly evolving agentic-AI field.	Broad scope; limited empirical isolation of coding failure-recovery mechanisms.
31	A Survey on Large Language Model Based Autonomous Agents [31]	2024	Unified survey of construction, capabilities, and applications of LLM autonomous agents.	Establishes broad conceptual foundation for LLM-agent research.	Not specific to repository-level coding recovery.
32	Vibe Coding vs. Agentic Coding: Fundamentals and Practical Implications of Agentic AI [32]	2025	Conceptual comparison of coding approaches and agentic coding practices.	Helps distinguish conventional AI-assisted coding from agentic coding.	Primarily conceptual/practical; limited controlled empirical recovery evaluation.
33	A Self-Improving Coding Agent [33]	2025	Investigates mechanisms through which coding agents can improve using previous experience.	Directly relevant to self-improvement in coding agents.	Self-improvement is broader than controlled single-round diagnosis-based recovery.
34	The 2025 AI Agent Index: Documenting Technical and Safety Features of Deployed Agentic AI Systems [34]	2026	Documents technical, capability, ecosystem, and safety characteristics of deployed agents.	Provides empirical documentation of the agent ecosystem and transparency.	Focuses on deployed-agent documentation rather than coding-recovery mechanisms.
35	SMART-LLM: Smart Multi-Agent Robot Task Planning Using Large Language Models [35]	2024	Uses multiple LLM-based agents for task planning and execution.	Demonstrates multi-agent planning and coordination.	Robotic task setting differs substantially from repository-level coding recovery.

3.9.1 Thematic Synthesis of the Literature

The reviewed studies can be organized into several major research themes.

Table 3.2: Thematic Classification of Reviewed Literature

Research Theme	Representative Studies	Main Focus
Autonomous Coding Agents	[3], [4], [5], [6]	Repository exploration, code modification, tool use, testing and autonomous repair
Self-Reflection	[7], [8], [9], [10], [12]	Critique, self-feedback, self-debugging and iterative revision
Memory and Experience	[11], [15], [16], [17]	Retaining and reusing previous experiences and information
Planning and Tool Use	[22], [23], [24], [25], [28]	Planning, API interaction and tool execution
Failure Analysis	[13], [21]	Error classification, diagnosis and automated feedback
Automated Program Repair	[3], [5], [10], [18]	Generating and validating software repairs
Evaluation Benchmarks	[19], [20], [27]	Measuring agent capabilities and reliability
General Agentic AI	[1], [2], [29], [30], [31], [32], [34], [35]	Agent architectures, applications and broader research directions

The thematic analysis shows that the literature contains substantial work on individual components of autonomous agent behaviour. However, these components are not uniformly evaluated together in the context of repository-level failure recovery.

3.10 Research Gap Identification

The literature reviewed above reveals several important observations.

3.10.1 Gap 1: Limited Controlled Comparison of Recovery Strategies

A significant portion of existing research demonstrates that reflection, feedback, debugging, or iterative execution can improve agent performance. Reflexion and Self-Refine, for example, demonstrate the potential of feedback-based refinement.

However, fewer studies directly compare:

Blind Retry vs. Reflection-only vs. Execution-Grounded Diagnosis

under the same repository-level task conditions.

The absence of such a controlled comparison makes it difficult to determine whether improvements originate from reflection itself, additional information, or simply additional attempts.

3.10.2 Gap 2: Diagnosis Does Not Necessarily Imply Recovery

Recent work such as AgentErrorTaxonomy and AgentDebug demonstrates that identifying and diagnosing agent failures can improve recovery in general agent environments.

However, repository-level coding introduces additional complexity.

A coding agent may receive a test failure but still fail to determine:

- which part of the generated code is incorrect;
- why the incorrect behaviour occurred;
- how the repository's architecture affects the bug;
- which code should be modified;
- whether the proposed modification actually addresses the root cause.

Therefore, the presence of diagnostic information does not necessarily mean that the agent can use that information successfully.

3.10.3 Gap 3: Limited Repository-Level Evaluation of Failure Recovery

SWE-bench provides a realistic environment for evaluating repository-level coding capabilities.

However, benchmark-level task success alone does not reveal the internal behaviour of an agent after a failed attempt.

A conventional benchmark evaluation may report:

Attempt → FAIL

without determining whether the failure resulted from:

- incorrect reasoning;
- incorrect file selection;
- incorrect patch representation;
- patch-application failure;
- execution-environment failure;
- test interpretation failure.

This creates a methodological challenge for failure-recovery research.

3.10.4 Gap 4: Patch Reliability and Reasoning Correctness Can Be Confounded

A coding agent can potentially reason correctly but still produce a patch that cannot be applied.

For example:

Correct diagnosis → Correct intended modification → Malformed patch → Evaluation FAIL

would be classified as a failure even though the underlying reasoning may have been correct.

Conversely:

Incorrect diagnosis → Valid patch format → Patch applies → Tests FAIL
represents a genuine reasoning failure.

Therefore, patch-formatting reliability and reasoning correctness represent conceptually distinct dimensions that should be separated during evaluation.

3.10.5 Gap 5: Limited Evidence on Single-Round Recovery

Many agentic systems use iterative execution, but the exact contribution of a **single recovery intervention** remains unclear.

The present research therefore isolates a recovery attempt and compares three controlled conditions:

1. **Blind Retry**
2. **Reflection-only**
3. **Diagnose+Revise**

This design allows the effect of additional information to be examined independently of simply allowing an unrestricted number of attempts.

3.10.6 Gap 6: Need for Robustness Checks

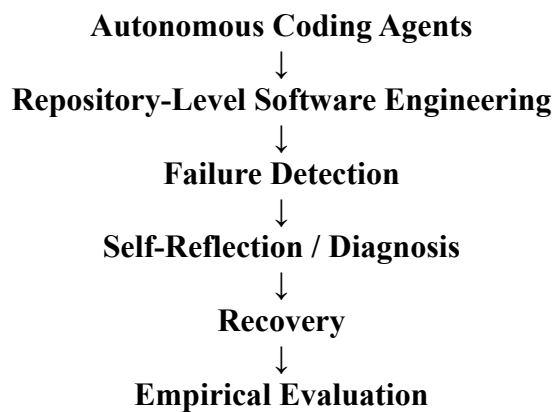
A result obtained from one experimental run may depend on model sampling, execution conditions, patch representation, or other implementation factors.

Therefore, the present study includes a second recovery round to determine whether the observed behaviour remains consistent.

This is particularly important when interpreting a null result. A failure to observe improvement should not immediately be interpreted as evidence that a recovery mechanism is ineffective in general.

3.11 Positioning of the Present Study

Based on the literature reviewed above, the present dissertation is positioned at the intersection of:



The study differs from many previous works by focusing specifically on the transition from **failure observation to recovery**.

The experimental design compares three recovery conditions:

Blind Retry

The agent receives the original task information and attempts another solution without structured additional diagnosis.

Reflection-only

The agent receives information about its previous failed attempt and is asked to reflect on what may have gone wrong before generating a revised solution.

Diagnose+Revise

The agent receives execution-grounded evidence from the failed attempt and is explicitly asked to diagnose the likely cause before generating a revised solution.

The conceptual distinction is therefore:

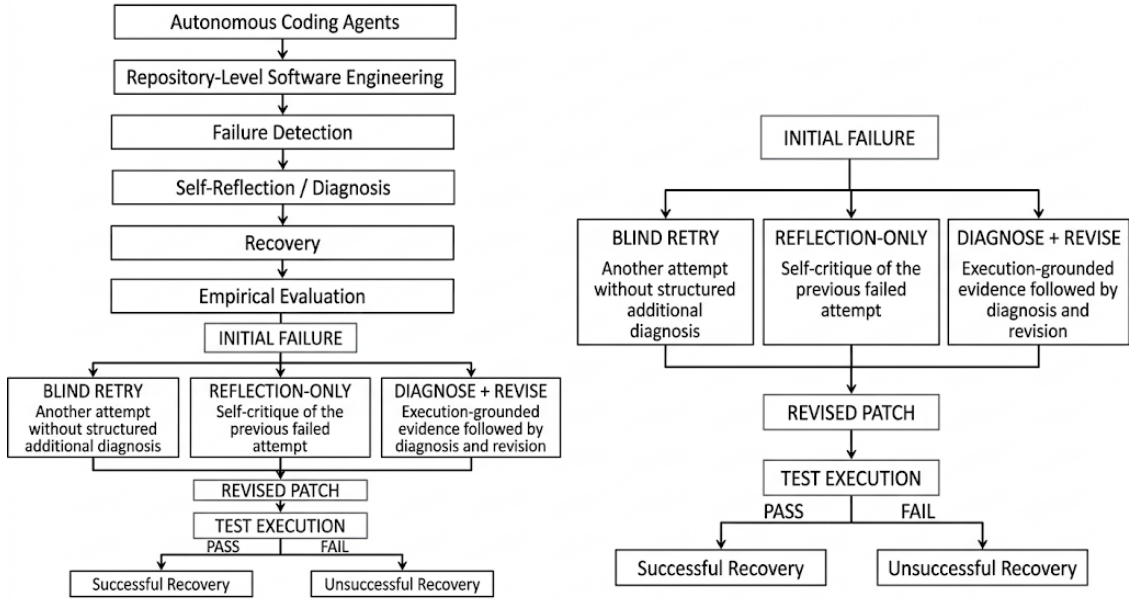


Figure 3.3 Positioning and Recovery Strategy Framework of the Present Study

The research consequently does not ask only whether an agent can generate a correct patch. Instead, it investigates whether **additional information available after failure changes the probability of successful recovery**.

3.12 Chapter Summary

This chapter reviewed the literature relevant to autonomous coding agents and failure recovery in LLM-based software engineering systems.

The literature demonstrates substantial progress in autonomous repository interaction, program repair, self-reflection, self-debugging, memory, planning, tool use, and agent evaluation. Systems such as SWE-agent, AutoCodeRover, RepairAgent, and OpenHands demonstrate the increasing capability of LLM-based agents to perform software engineering tasks.

Research on Reflexion, Self-Refine, self-debugging, and automated feedback demonstrates that iterative reasoning and feedback can improve LLM behaviour in several settings.

Research on memory and experiential learning further suggests that agents can potentially use previous experiences to improve future behaviour.

At the same time, recent failure-analysis research demonstrates the importance of identifying root causes rather than simply observing that an agent has failed.

However, the literature leaves an important empirical question insufficiently addressed: **whether providing additional reflection or execution-grounded diagnostic information actually improves recovery from failed repository-level coding attempts when compared under controlled conditions with a simple retry strategy.**

A second methodological issue concerns the separation of **patch-formatting/application failures from genuine reasoning failures**. If these two failure types are not distinguished, an evaluation may incorrectly attribute a technical patch-generation problem to the reasoning capability of the coding agent.

The identified gaps therefore motivate the empirical investigation presented in Chapter 4. The next chapter describes the experimental methodology used to compare Blind Retry, Reflection-only, and Diagnose+Revise recovery strategies on repository-level software engineering tasks from SWE-bench Lite.

CHAPTER 4

RESEARCH METHODOLOGY

4.1 Experimental Overview

This chapter describes the experimental methodology used to compare three failure-recovery strategies for large language model (LLM)-based repository-level code repair. The chapter specifies the benchmark, the task sample, the model and access configuration, the manner in which repository context was prepared, the representation used to express code modifications, and the infrastructure used to verify each generated modification against the original project test suite. Sections 4.1 to 4.9 establish the experimental setup common to all conditions; Sections 4.10 to 4.19 define the individual recovery strategies, the outcome classification scheme, and the controls applied to the comparison.

The experiment follows a two-stage design. In the first stage, a single code modification is generated for each task in the sample and evaluated against the project test suite. Tasks that are not resolved at this stage form the failure pool. In the second stage, each task in the failure pool is submitted to three distinct recovery strategies, which differ only in the information made available to the model after the initial failure. The strategies are Blind Retry, in which the task is attempted again with no additional information; Reflection-only, in which the model receives its own previous attempt together with a request to critique it; and Diagnose+Revise, in which the model additionally receives execution evidence extracted from the failed test run. Because the three conditions differ in exactly one respect, namely the post-failure information supplied, any difference in outcome is attributable to that information.

A precise statement of the system under evaluation is necessary in order to avoid overstating its capabilities. The system implemented in this study performs a single, stateless model invocation per experimental cell. It does not navigate the repository, does not invoke external tools during generation, does not execute code as part of its reasoning process, and does not maintain state across invocations. The target file for each task is identified in advance from benchmark metadata rather than discovered by the system. The system is therefore accurately described as single-shot, oracle-localized, single-file patch generation, and the term agentic is used in this dissertation only in reference to prior systems such as SWE-agent, AutoCodeRover, RepairAgent, and OpenHands, which do incorporate tool use and iterative execution [3]-[6]. This restriction is deliberate. The objective of the study is to isolate the effect

of post-failure information on revision quality, and a simpler system permits that effect to be observed without confounding from repository navigation or tool-use behaviour.

Figure 4.1 presents the complete experimental pipeline. Each task enters the pipeline as a natural-language issue description together with repository metadata. The target source file is retrieved at the recorded base commit and supplied to the model in full. The model returns a set of textual edit instructions, which are applied programmatically to produce a modified file, from which a unified diff is then computed mechanically. The resulting patch is evaluated inside an isolated container against the project test suite, and the outcome is assigned to one of five categories defined in Section 4.14. Tasks that do not resolve re-enter the pipeline under each of the three recovery strategies.

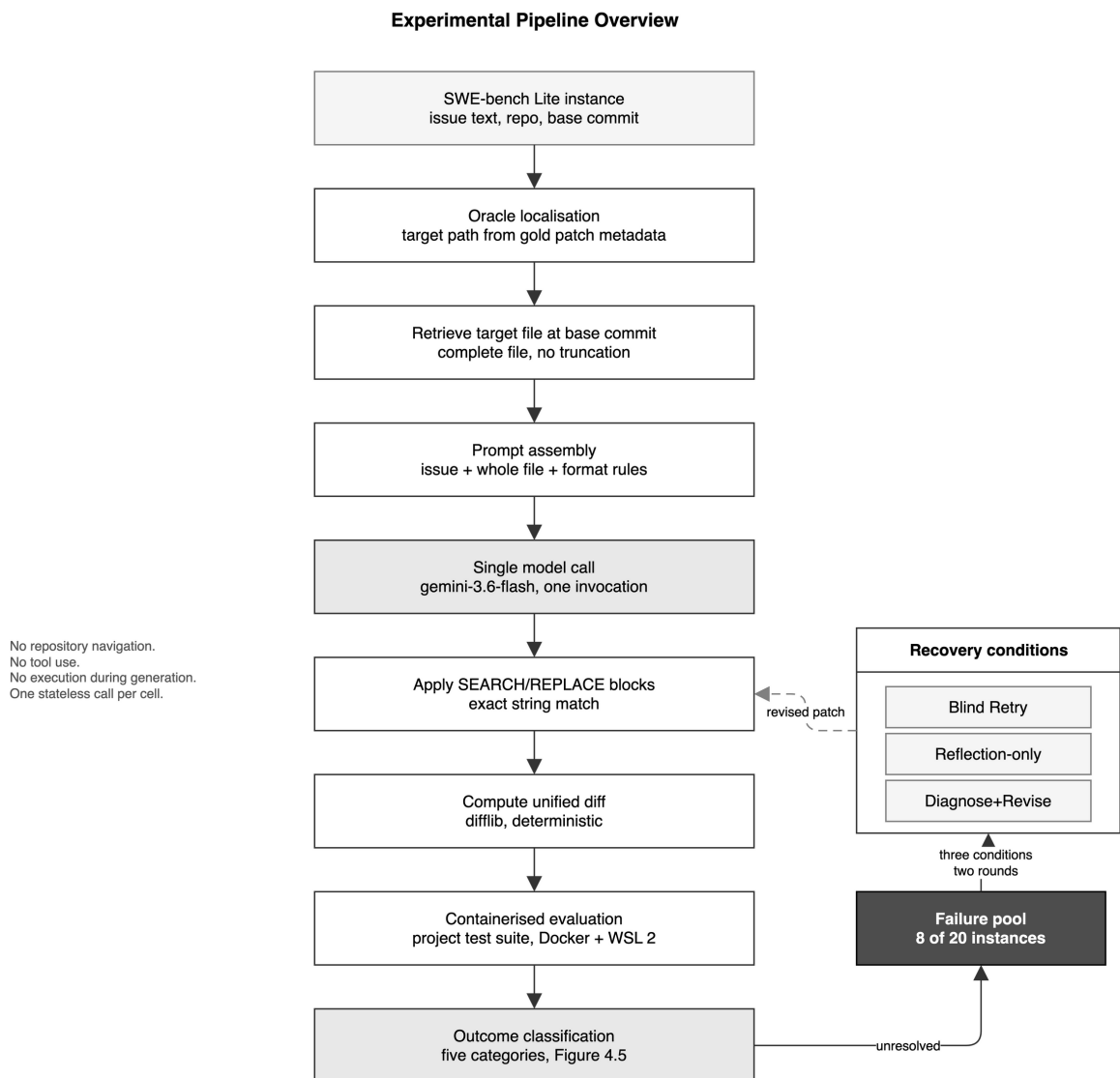


Figure 4.1 Experimental Pipeline Overview

4.2 Research Questions and Evaluation Design

The empirical component of this dissertation addresses the research questions stated in Section 2.3. Three of those questions are answered by the experiment described in this chapter, and the design of the experiment follows directly from their requirements.

Research Question 3 asks whether the availability of reflection or execution-grounded diagnostic information improves recovery from failed coding attempts. Answering this question requires a comparison in which the post-failure information is the only quantity that varies. The experiment therefore holds the model, the sampling configuration, the task sample, the repository context, the patch representation, and the evaluation harness constant across all three conditions, and varies only the content of the prompt supplied after a failure. Section 4.17 documents the controls applied and their limitations.

Research Question 4 asks to what extent observed recovery failures can be attributed to reasoning limitations rather than to patch-formatting or patch-application mechanisms. Answering this question requires that the two categories of failure be distinguishable in the recorded data. The experiment therefore classifies every baseline failure by whether the generated patch was successfully applied before any test was executed. A patch that cannot be applied produces no information about the reasoning that produced it, and pooling such cases with genuine test failures would confound a mechanical defect with a cognitive one. Section 4.14 defines the classification scheme, and Section 4.15 describes the corrective measure applied once the extent of the formatting problem became apparent.

Research Question 5 asks whether the observed recovery behaviour is consistent across repeated evaluation rounds. Answering this question requires that each condition be applied more than once to the same failure pool. The experiment therefore incorporates a second recovery round, described in Section 4.16, in which each strategy is granted a further opportunity to revise its solution using the evidence produced by its own preceding attempt.

Research Questions 1 and 2 are addressed by the systematic review presented in Chapter 3 and by the strategy definitions in Sections 4.10 to 4.13 respectively, and do not require additional experimental apparatus.

The dependent variable throughout is the outcome of executing the project test suite against the modified repository. The benchmark reports this outcome as a binary resolution flag. As Section 4.14 explains, the present study additionally records the underlying sub-test

counts, because a binary flag cannot represent partial progress and, in this experiment, conceals the largest single change observed in the data.

4.3 Dataset and Benchmark

The experiment uses SWE-bench Lite, a curated subset of the SWE-bench benchmark for repository-level software engineering [1]. SWE-bench is constructed from resolved issues and their corresponding pull requests in widely used open-source Python projects. Each benchmark instance provides a natural-language issue description, the repository identifier, the base commit at which the issue was present, the reference modification authored by the project maintainers, and two sets of tests. SWE-bench Lite comprises 300 instances drawn from 11 repositories and was released in order to reduce the computational cost of evaluation relative to the full benchmark of 2,294 instances.

The two test sets are central to the evaluation. The FAIL_TO_PASS set contains tests that fail at the base commit and pass once the reference modification is applied; these tests define the required behaviour and a candidate modification must make all of them pass. The PASS_TO_PASS set contains tests that pass both before and after the reference modification; these tests guard against regression and a candidate modification must not cause any of them to fail. A benchmark instance is recorded as resolved only when every FAIL_TO_PASS test passes and no PASS_TO_PASS test regresses. This conjunction is important for the present study because it means a single regression is sufficient to record a failure irrespective of how much required behaviour was correctly implemented.

SWE-bench Lite was selected for three reasons. First, its instances are drawn from genuine project histories rather than synthesised, so the tasks exhibit the dependency structure, implementation constraints, and test coverage of production software. Second, evaluation is fully automated and deterministic given a patch, which removes subjective judgement from the dependent variable. Third, the benchmark is widely used in the agentic coding literature reviewed in Chapter 3, which permits the baseline resolution rate reported in Chapter 5 to be interpreted against published results.

The benchmark is distributed through the HuggingFace datasets library. Generation scripts load the dataset under the identifier `princeton-nlp/SWE-bench_Lite`, and the evaluation harness is invoked with the equivalent identifier `SWE-bench/SWE-bench_Lite`. These are two aliases for the same distribution, and the test split is used throughout.

One property of the benchmark constrains the interpretation of all results reported in this dissertation. The repositories from which SWE-bench is constructed are public, and the commits from which the instances are derived predate the training cut-off of any contemporary language model. Reference solutions may therefore be present in the training data of the model under evaluation. This study observed a direct provider-side indication that this is so, and the matter is reported in Section 4.18 and carried into the validity analysis in Section 5.12.

4.4 Task Selection

The experiment uses the first 20 instances of the SWE-bench Lite test split, selected by their position in the distributed ordering. The sample is therefore a contiguous prefix of the benchmark rather than a random draw. Table 4.1 lists the selected instances together with the outcome each received at the baseline stage.

The sample size was determined by the access constraints documented in Section 4.5. A pilot of 20 instances was adopted because it was expected to yield a failure pool large enough to support a three-way comparison while remaining within the daily request ceiling available to the study. That expectation was met: 8 of the 20 instances did not resolve at the baseline stage, and those 8 constitute the failure pool used throughout Sections 4.10 to 4.16.

Three properties of this sample restrict the generality of the findings and are recorded here rather than deferred, because they bear on how the results in Chapter 5 should be read.

4.4.1 The Sample Is Not Random

Because the sample is a contiguous prefix of an ordered distribution rather than a random draw, it cannot be treated as representative of SWE-bench Lite. No claim in this dissertation is generalised to the full benchmark on the basis of this sample. A stratified random sample would be required for such a claim, and is identified as future work in Section 6.6.

4.4.2 Repository Coverage Is Narrow

The 20 selected instances are drawn from 2 of the 11 repositories represented in SWE-bench Lite. Of these, 14 instances originate from the Django web framework and 6 from the astropy astronomy library. The ordering of the distributed dataset groups instances by repository, so a contiguous prefix necessarily concentrates on the first repositories in that ordering. Results are consequently conditioned on the code style, test conventions, and module structure of two projects, and the predominance of Django means the sample is weighted towards a single codebase.

4.4.3 Task Solvability Was Not Independently Audited

The human screening exercise conducted during the construction of SWE-bench Verified found that a substantial proportion of SWE-bench instances are problematic as stated: issue descriptions that underspecify the required behaviour, and tests that impose requirements not derivable from the issue text. Instances of this kind cannot be solved reliably from the information supplied, irrespective of model capability. The present study did not conduct an independent solvability audit of the four logic-class failures in its pool, and the possibility that one or more of them is underspecified or unfairly tested is therefore retained as a threat to validity in Section 5.12. This matters specifically for the interpretation of a null result: a strategy cannot be shown ineffective on a task that no strategy could solve.

Table 4.1: Task Selection — 20 Pilot Instances

#	Instance identifier	Repository	Baseline outcome
1	astropy__astropy-6938	astropy	Resolved
2	astropy__astropy-7746	astropy	Unresolved (logic)
3	astropy__astropy-12907	astropy	Resolved
4	astropy__astropy-14182	astropy	Not applied (malformed)
5	astropy__astropy-14365	astropy	Resolved
6	astropy__astropy-14995	astropy	Resolved
7	django__django-10914	django	Resolved
8	django__django-10924	django	Resolved
9	django__django-11001	django	Resolved
10	django__django-11019	django	Unresolved (logic)
11	django__django-11039	django	Not applied (malformed)
12	django__django-11049	django	Resolved
13	django__django-11099	django	Resolved
14	django__django-11133	django	Resolved
15	django__django-11179	django	Resolved
16	django__django-11283	django	Unresolved (logic)
17	django__django-11422	django	Resolved
18	django__django-11564	django	Unresolved (logic)
19	django__django-11583	django	Not applied (malformed)
20	django__django-11620	django	Not applied (malformed)

The baseline outcomes summarised in Table 4.1 are 12 resolved instances, 4 instances in which the patch applied but the required tests did not pass, and 4 instances in which the generated patch could not be applied at all. The significance of the distinction between the

latter two groups is developed in Sections 4.8 and 4.14 and is the subject of Research Question 4.

4.5 Model and Access Configuration

All code modifications evaluated in this study were generated by a single model, gemini-3.6-flash, accessed through the Google generative AI Python SDK. A single model was used throughout so that differences between conditions could not be attributed to model substitution. Table 4.2 records the complete configuration.

Table 4.2: Model and Access Configuration

Parameter	Value
Model identifier	gemini-3.6-flash
Software development kit	google-genai (Python)
Access tier	Google AI Studio, free tier
Sampling temperature	Not specified; SDK default applied
Nucleus sampling (top-p) and top-k	Not specified; SDK default applied
Random seed	Not specified
Maximum output tokens	Not specified
Model invocations per experimental cell	1
Inter-request delay	60 seconds
Daily request ceiling	20 requests
Benchmark	SWE-bench Lite, test split, first 20 instances
Evaluation harness	swebench.harness.run_evaluation
Harness parallelism	2 workers
Container runtime	Docker Desktop with WSL 2 integration
Host platform	Windows with Windows Subsystem for Linux 2

4.5.1 Sampling Configuration

No sampling parameters were specified in any generation script. Temperature, nucleus sampling, top-k, and random seed were left unset, and the corresponding software development kit defaults were applied implicitly. Each generation is therefore a single draw from a sampling distribution that was not pinned by the experimenter. Two consequences follow and are stated plainly.

First, exact reproduction of an individual experimental cell cannot be guaranteed. The pipeline is fully scripted and every script, prompt, prediction file, and evaluation log is

published, so the procedure is auditable and re-executable; but re-execution may not return the identical patch. The claim that can be supported is that the pipeline is reproducible as a procedure, not that individual generations are deterministic. Section 4.17 returns to this point.

Second, and in the opposite direction, any stability observed across independent generations is stronger evidence than it would be under a fixed seed. Where Chapter 5 reports that a task returned an identical sub-test score across six separate model invocations spanning three strategies and two rounds, that agreement was obtained from six unpinned draws rather than from a single seeded draw repeated six times. Such a result constrains the behaviour of the system more tightly than a deterministic replication would.

4.5.2 Access Constraints

The study was conducted without funding, using free-tier access. The applicable quota for this model permits 20 generation requests per day, and requests beyond that ceiling are refused with an explicit resource-exhaustion response. A 60-second delay was inserted before each request in order to remain within the per-minute rate limit.

This constraint shaped the experiment and is therefore reported as part of the methodology rather than confined to a limitations section. A single complete recovery round over the failure pool of 8 instances under 3 strategies requires 24 generation requests. The daily ceiling is 20. It was consequently not possible to evaluate every combination of instance, strategy, and round within a single day, and the evaluation was distributed across multiple days with some combinations left unevaluated. The resulting coverage is reported in full in Chapter 5, where every unevaluated combination is marked as unevaluated rather than treated as a negative result. Section 4.18 discusses the practical implications of conducting the study under this constraint, and Section 6.6 identifies the migration to a higher-throughput provider as the first priority for any extension of this work.

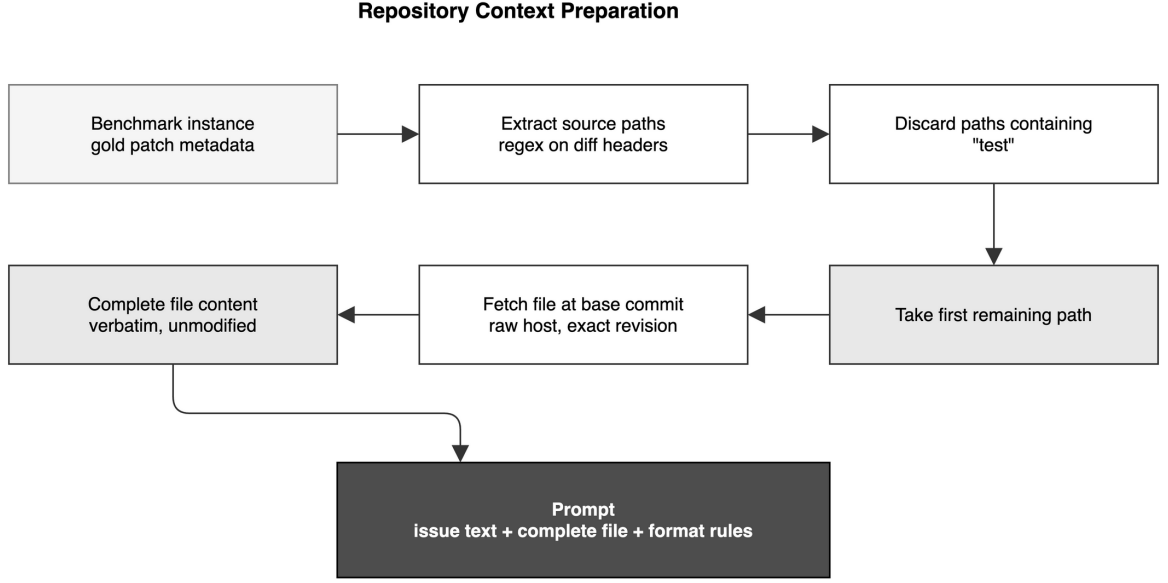
4.6 Repository Context and Task Preparation

Each benchmark instance identifies a repository and a base commit at which the reported issue is present. For every task the target source file was retrieved directly from the hosting service at that exact commit, ensuring that the model received the file in precisely the state in which the issue existed rather than in its current or subsequently modified state. Retrieval at the recorded commit is essential: a later revision of the file might already contain the fix, or might have been restructured such that the issue description no longer applies.

The retrieved file was supplied to the model in full. No summarisation, truncation, chunking, embedding-based retrieval, or relevance filtering was applied to the source context. The complete file was interpolated verbatim into the prompt, accompanied by the issue description and the instructions governing the required output format. Figure 4.2 illustrates this preparation sequence.

Supplying the entire file removes an important source of variability. Had a retrieval or summarisation component been interposed, a failure could have originated in that component rather than in the model's reasoning, and the comparison between recovery strategies would have been confounded by differences in what each strategy happened to retrieve. Providing the whole file guarantees that all three conditions operate on identical source context, so that the only quantity differing between them is the post-failure information under study.

This choice has a substantial and quantifiable cost. The target files in the sample are production source files of considerable size. The largest encountered, the world coordinate system module of `astropy` required by instance `astropy__astropy-7746`, contains 124,394 characters, corresponding to approximately 31,100 tokens. That volume is transmitted in its entirety on every request concerning that instance. No prompt caching or context reuse was employed, so a single task evaluated under three strategies retransmits the same file three times, and each additional recovery round retransmits it again. Because the free-tier quota described in Section 4.5 is expressed in requests rather than tokens, this cost did not reduce the number of tasks that could be attempted; it does, however, establish the cost baseline against which the relative expense of the three strategies is assessed in Section 5.7, and it would become the dominant constraint under any token-metered access arrangement.



Only the PATH is taken from the gold patch. Its content never enters any prompt.
 Cost: the largest target file in the sample, `astropy/wcs/wcs.py`, is 124,394 characters, approximately 31,100 tokens, retransmitted in full on every request. No prompt caching was used.

Figure 4.2 Repository Context Preparation

4.7 Oracle-Based File Localization

Repository-level code repair is conventionally decomposed into two sub-problems: identifying which file or files require modification, and determining the modification itself. The present study addresses only the second. The file to be modified is supplied to the system rather than discovered by it, a procedure commonly termed oracle localization.

The target path was obtained from benchmark metadata. Each instance includes the reference modification authored by the project maintainers, expressed as a unified diff. The file paths in that diff were extracted by matching the source-file header lines, and paths containing the substring `test` were discarded so that test files introduced by the reference modification were not mistaken for implementation targets. Where more than one implementation file remained, the first was selected. Only the path was taken from the reference modification; its content was never supplied to the model, and no part of the reference solution entered any prompt.

Two implications require explicit statement.

4.7.1 The Evaluated System Is Not Performing Localization

Because the target file is supplied, the resolution rates reported in Chapter 5 are not comparable with those of systems that perform their own localization, such as SWE-agent,

AutoCodeRover, or OpenHands [3]-[6]. Those systems solve a strictly harder problem. The figures in this dissertation should be read as measurements of repair quality conditional on correct localization, and Chapter 5 does not compare them against published end-to-end resolution rates.

4.7.2 Oracle Localization Strengthens a Null Result

The principal empirical finding of this study, presented in Chapter 5, is that additional post-failure information did not improve recovery on tasks whose failure was genuinely one of program reasoning. Oracle localization makes that finding harder to dismiss rather than easier. Under a system performing its own localization, an unimproved outcome could be attributed to the model having examined the wrong file, and the result would be uninformative about reflection or diagnosis. In the present design the model was given the correct file on every attempt, and the complete contents of that file, and in the Diagnose+Revise condition the output of the failing tests as well. The information required to identify the defect was demonstrably present. That the additional information produced no improvement therefore constitutes evidence about the use the model made of it, and not about whether it was available.

One structural limitation follows from the selection rule. Where a reference modification spans several implementation files, only the first was supplied, so the model could not express a change requiring coordinated edits across files. No instance in the failure pool was found to require such a change, but the constraint is recorded because it bounds the class of task the system is in principle capable of resolving, and it is revisited in Section 5.12.

4.8 Patch Representation and Application

A generated modification must be converted into a patch that the evaluation harness can apply. This apparently mechanical step proved to be methodologically decisive, and the manner in which it was implemented changed during the study. Both implementations are described here, because results obtained under them are not comparable and are reported separately throughout Chapter 5.

4.8.1 Initial Approach: Model-Authored Unified Diff

In the initial implementation the model was asked to emit a unified diff directly. This is the format the evaluation harness consumes, so the output could be forwarded without intermediate processing.

The approach proved unreliable. A unified diff is a positional format: each hunk header declares the line at which the change begins and the number of lines of context on either side, and the patch is rejected if these do not correspond exactly to the target file. Producing a correct hunk header requires accurate line counting over a file of many thousands of lines, which is a clerical operation poorly matched to a generative model. Of the 20 baseline instances, 4 produced patches that could not be applied at all, the harness reporting in one case that only unusable content was present in the patch input. In each of these cases no test was ever executed, and consequently no information whatsoever was obtained about whether the intended modification was correct.

This is a confound rather than a mere inconvenience. A benchmark that records such an instance as a failure attributes to the model's reasoning a defect that lies entirely in the transcription of the edit. Four of the eight members of the failure pool, one half, entered it for this reason. Any comparison of recovery strategies conducted over that pool without separating the two categories would have measured the strategies' capacity to produce well-formed diffs rather than their capacity to repair software.

4.8.2 Revised Approach: Search-and-Replace Blocks

The representation was therefore changed. In the revised implementation the model is asked to express each edit as a pair of literal text blocks: the exact existing text to be located, and the text that is to replace it. The model is not required to count lines, compute offsets, or construct hunk headers. The edit is instead applied programmatically by locating the search text within the retrieved file and substituting the replacement, and the unified diff required by the harness is then computed mechanically from the original and modified file contents using the standard library `diff` module. The diff is thus generated by a deterministic procedure that cannot produce a malformed hunk header, and the model's responsibility is reduced to identifying the correct text to change.

The applicator enforces an exact-match requirement with three rejection conditions, shown as terminal states in Figure 4.3. If the search text occurs exactly once, the substitution is performed. If it does not occur, the edit is rejected as not found. If it occurs more than once, the edit is rejected as ambiguous, because the intended location cannot be determined. If the response contains no parsable blocks, it is rejected as unparsable. In each rejection case no patch is produced.

The behaviour following a rejection must be recorded precisely, because it affects how the results tables in Chapter 5 are to be read. A rejected response was reported to the console and then discarded: no entry was written to the predictions file for that task, the evaluation harness was never invoked for it, and no evaluation record was produced. The corresponding experimental cell is therefore absent from the data rather than present with a negative outcome. Together with the request ceiling described in Section 4.5, this is one of two mechanisms by which combinations of task, strategy, and round came to be unevaluated. Chapter 5 marks every such combination as unevaluated and never treats an absent cell as evidence that a strategy failed.

The change of representation is referred to in this dissertation as the fairness retrofit, because its purpose is to ensure that the three strategies are compared on their reasoning rather than on their facility with a positional text format. Section 4.15 documents the retrofit as an experimental control, and Section 5.3 reports the effect it had on the measured outcomes, which was substantial.

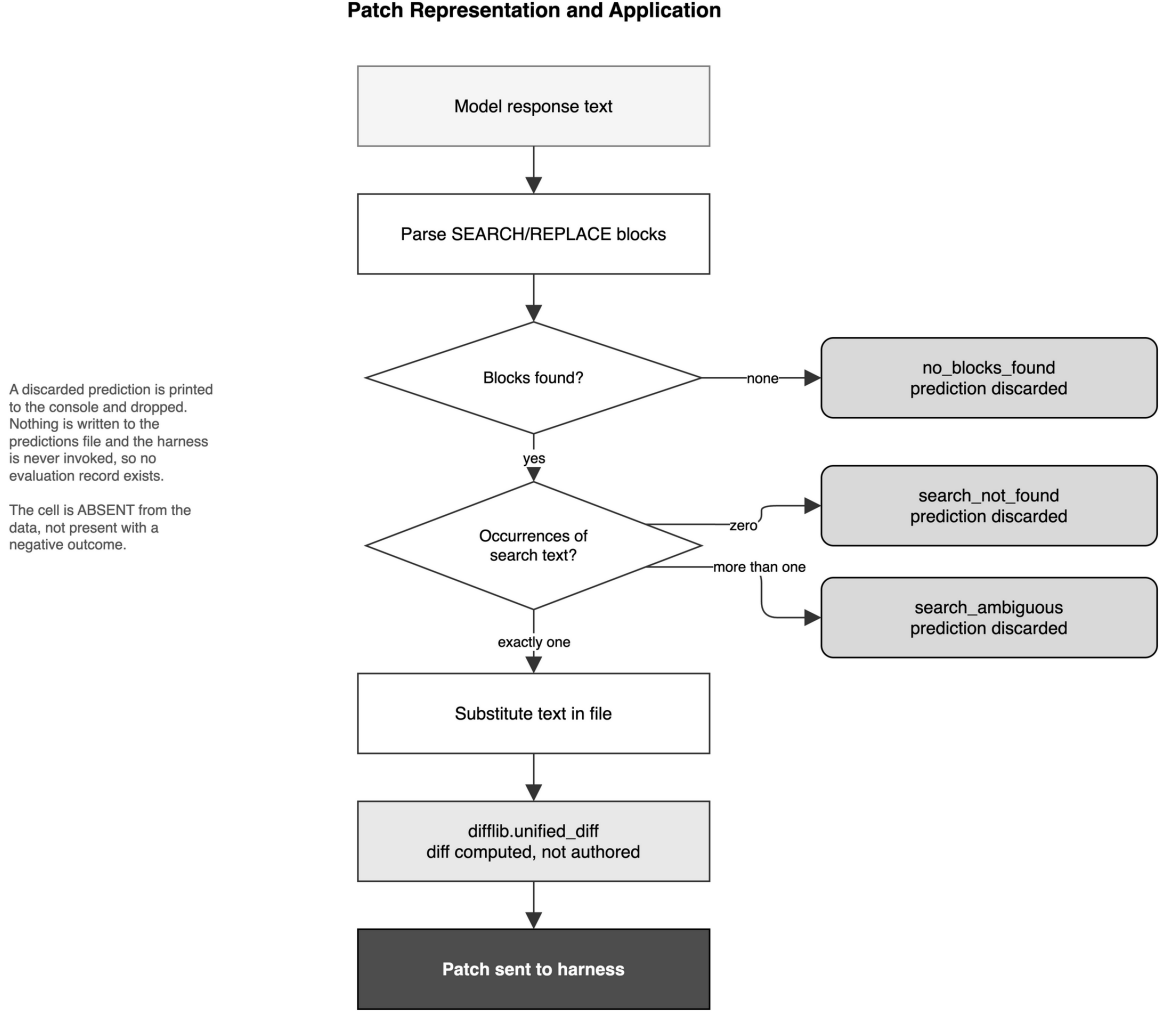


Figure 4.3 Patch Representation and Application

4.9 Evaluation Infrastructure

Every candidate modification was verified by execution. No outcome reported in this dissertation rests on inspection of a patch, on model self-assessment, or on similarity to the reference solution; each rests on the observed result of running the project's own test suite against the modified repository.

Verification used the official SWE-bench evaluation harness, invoked as the `run_evaluation` module of the `swebench` package. For each task the harness constructs an isolated container image containing the repository at the recorded base commit together with the dependency set required by that project and commit, applies the candidate patch, executes the designated `FAIL_TO_PASS` and `PASS_TO_PASS` tests, and writes a structured report recording whether the patch applied, the status of each individual test, and the resulting

resolution flag. The harness was run with two parallel workers, a setting chosen to remain within the memory available on the host.

Containerisation is essential to the validity of the comparison. The repositories in the benchmark are evaluated at commits several years old and require dependency versions that conflict both with one another and with any contemporary development environment. Executing each task in an image constructed for its own commit ensures that a test failure reflects the candidate patch and not an incompatibility in the surrounding environment, and that every condition is evaluated under identical circumstances.

The harness was executed on a Windows host through the Windows Subsystem for Linux, with container support provided by Docker Desktop configured for integration with that subsystem. Two infrastructure problems were encountered and resolved during the study, and are recorded here because they are likely to recur for anyone reproducing this work. The first concerns line endings: patches authored on the Windows host carried carriage-return line terminators, which the patch utility inside the Linux container rejected. The second concerns container reachability, the container daemon becoming unreachable from the subsystem until integration was enabled in the runtime's resource settings and the runtime restarted. Neither problem affected any recorded result, since both manifested as an inability to complete an evaluation rather than as an incorrect outcome, but both consumed time that the request ceiling had already made scarce.

The structured report produced for each evaluation is the primary record of this study. All results presented in Chapter 5 are derived from these reports by script, and the derivation can be re-executed against the published logs at any time. In particular, the sub-test counts used in Sections 5.4 and 5.7 are read from the per-test status fields of these reports rather than from the summary resolution flag, for the reasons given in Section 4.14.

4.10 Baseline Evaluation

The baseline stage establishes which tasks fail, and in what manner, before any recovery strategy is applied. Each of the 20 selected instances received exactly one generation attempt. The prompt supplied the repository name, the issue description recorded in the benchmark, the complete contents of the target file retrieved at the base commit, and the instructions governing the required output format. No information about the reference solution, the test names, or the expected behaviour beyond the issue text was provided.

The resulting patches were evaluated by the harness described in Section 4.9. The outcome distribution was 12 instances resolved, 4 instances in which the patch applied but the required tests did not pass, and 4 instances in which the generated patch could not be applied at all. The 8 unresolved instances constitute the failure pool, and every recovery condition described below operates on that pool. The complete baseline record appears as Table 5.1.

Two features of the baseline require emphasis before the recovery strategies are defined. The first is that the failure pool is not homogeneous. Half of its members entered it because no patch could be applied, which means no test was executed and no evidence was obtained about the quality of the underlying reasoning. The other half entered it because a validly applied patch did not produce the required behaviour. These are different kinds of failure and, as Section 4.14 sets out, they are classified separately and reported separately throughout Chapter 5.

The second is that the baseline attempt is itself the first element of the comparison. Blind Retry, defined in Section 4.11, is a second attempt made under the same conditions as the first. The baseline therefore functions both as the selection mechanism for the failure pool and as the reference point against which the effect of additional post-failure information is measured.

4.11 Recovery Strategy 1: Blind Retry

Blind Retry re-attempts a failed task with no information about the failure. The prompt is constructed identically to the baseline prompt: the repository name, the issue description, the complete target file, and the output-format instructions. The model is not informed that a previous attempt was made, is not shown that attempt, and receives no test output. Any change in outcome relative to the baseline is therefore attributable to sampling variation alone.

This condition is essential to the design rather than merely a control of convenience. A recovery strategy that supplies additional information can only be credited with an improvement if that improvement exceeds what re-sampling achieves unaided. Much of the reported benefit of iterative self-correction in the literature has been obtained without such a control, and Chapter 3 identifies its absence as one of the gaps this study addresses. Blind Retry establishes the floor against which the remaining two conditions are assessed.

4.12 Recovery Strategy 2: Reflection-only

Reflection-only supplies the model with its own previous attempt and requires it to criticise that attempt before proposing a revision. The prompt states that a previous attempt was made and did not resolve the issue, reproduces the complete text of that previous response, and

instructs the model to reflect critically on what might have been wrong with its earlier reasoning before providing a revised fix. The response format is extended by a REFLECTION field of two to four sentences that precedes the explanation and the edits.

The previous attempt is supplied as the complete raw text of the earlier model response, comprising its stated explanation and its proposed edits, rather than as the derived patch. The model therefore sees its own reasoning as it originally expressed it, which is the form of self-reflection described in the Reflexion and Self-Refine literature reviewed in Sections 3.3.1 and 3.3.2.

Critically, no execution information is provided. The model is told that its previous attempt failed, but not which tests failed, nor with what error, nor whether the failure arose from incorrect logic or from an unmet requirement it had not considered. The condition therefore tests whether unaided introspection over an unsuccessful attempt is sufficient to improve it, which is the precise question raised by Huang et al. in their analysis of intrinsic self-correction.

4.13 Recovery Strategy 3: Diagnose+Revise

Diagnose+Revise supplies everything given to Reflection-only and adds the observed result of executing the previous patch against the project test suite. The prompt presents this material explicitly as ground truth rather than conjecture, and instructs the model to diagnose the specific root cause on the basis of that evidence before revising. The response format substitutes a DIAGNOSIS field for the REFLECTION field, requiring the identified root cause to be grounded in the evidence supplied.

The evidence is extracted from the artefacts written by the evaluation harness during the failed run. Two kinds are distinguished. Where the patch applied and tests executed, the captured test output is used, labelled as the real result of running the previous patch. Where the patch could not be applied, the patch application error recorded in the instance log is used instead, labelled as an indication that the previous patch could not be applied at all. The distinction matters because the two convey different information: the former describes incorrect behaviour, the latter describes a malformed edit.

The evidence is truncated before insertion. Test logs produced by these projects are large, the largest encountered in this study approaching 1.7 megabytes, and cannot be supplied whole. Where the evidence exceeds 3,000 characters, only the final 3,000 are retained, preceded by a marker indicating that earlier output was removed. Retaining the tail is defensible

for Python test output, since the exception type, the failing assertion, and the summary line appear at the end. It is nonetheless a substantial reduction: for the largest log encountered it retains approximately 0.2 per cent of the output, and in a run where several tests fail the earlier failures are discarded. This bound is a property of the condition as implemented, and Section 5.8 takes it into account when interpreting why execution-grounded diagnosis did not outperform the simpler strategies.

4.13.1 Comparability of the Three Conditions

The three prompts are strictly nested. The Blind Retry prompt is a subset of the Reflection-only prompt, which is in turn a subset of the Diagnose+Revise prompt. The shared portion, comprising the role statement, the issue description, the complete target file, and the output-format instructions, is identical in wording across all three conditions. The conditions differ only by the addition of the previous attempt, and then of the execution evidence.

This nesting is what licenses the comparison. Because each condition adds information to the one before it without altering anything already present, any difference in outcome is attributable to the added information rather than to incidental differences in phrasing, ordering, or task framing. Figure 4.4 presents the three conditions side by side and identifies exactly what enters the prompt in each, and Table 4.3 states the same relationship in tabular form. The independent variable of this study is the content of that figure.

Information Available to Each Recovery Strategy

Supplied to the prompt	Blind Retry	Reflection-only	Diagnose+Revise
Issue description	supplied	supplied	supplied
Complete target file	supplied	supplied	supplied
Output format rules	supplied	supplied	supplied
Told previous attempt failed	withheld	supplied	supplied
Text of previous attempt	withheld	supplied	supplied
Must critique own reasoning	withheld	supplied	supplied
Observed test output	withheld	withheld	supplied
Must diagnose root cause	withheld	withheld	supplied
Relative input cost	1x	2x	3x

The three prompts are strictly nested: the shared portion is identical in wording, and each condition adds material without altering anything already present. Shaded cells are supplied to the condition; dashed cells are withheld. This figure is the independent variable of the study — every difference in outcome reported in Chapter 5 is attributable to the rows that differ between these columns.

Figure 4.4 Information Available to Each Recovery Strategy

Table 4.3: Recovery Strategy Definitions

Information supplied	Blind Retry	Reflection-only	Diagnose+Revise
Issue description	Yes	Yes	Yes
Complete target file	Yes	Yes	Yes
Output-format instructions	Yes	Yes	Yes
Told that a previous attempt failed	No	Yes	Yes
Text of the previous attempt	No	Yes	Yes
Required to critique its own reasoning	No	Yes	Yes
Observed test output from the failed run	No	No	Yes
Required to diagnose a root cause	No	No	Yes
Approximate input cost relative to Blind Retry	1x	2x	3x

The final row of Table 4.3 records the cost of each condition, expressed as a multiple of the input volume required by Blind Retry. Because the complete target file is retransmitted on every request and the added material is small by comparison for most tasks, the ratio is dominated by the number of large blocks each prompt carries. This quantity is reported alongside every effectiveness measure in Section 5.7, for the reason given in Section 4.18.4: a strategy that returns an identical outcome at three times the cost is a result in its own right, and the recovery literature has tended to report effectiveness without it.

4.14 Metrics and Outcome Classification

The benchmark reports a single binary flag per instance, recording whether the instance was resolved. That flag is retained as the primary measure, because it is the measure used in the published literature and comparability requires it. It is insufficient on its own, for two reasons established during this study.

4.14.1 Sub-Test Scoring

The binary flag cannot represent partial progress. An instance requiring sixteen tests to pass is recorded identically whether none of them passes or fifteen do. In this study that limitation is not hypothetical: the largest single change observed in the entire dataset occurs on an instance that the binary flag records as unresolved both before and after the change. Reporting only the flag would have concealed it entirely.

This study therefore records, for every evaluated cell, the number of FAIL_TO_PASS tests that passed out of the number required, and the number of PASS_TO_PASS tests that regressed. These counts are read from the per-test status fields of the harness report rather than

from its summary flag. The practice follows recent work arguing that aggregate resolution rates conceal the greater part of what an evaluation observes, and reporting it constitutes part of this study's methodological contribution under Objective 8.

4.14.2 Unevaluated Cells

A classification scheme confined to test outcomes cannot describe a cell for which no evaluation exists. As Sections 4.5 and 4.8 establish, two mechanisms produced such cells: the daily request ceiling, which made some combinations impossible to attempt on the day they were scheduled, and the rejection of an unmatched or ambiguous edit, after which no patch was written and the harness was never invoked. In both cases the cell is absent from the data rather than present with a negative outcome. The taxonomy below therefore includes an explicit category for such cells, and Chapter 5 marks them as unevaluated in every table. Treating an absent cell as a failure would attribute to a strategy an outcome it was never given the opportunity to produce.

4.14.3 The Outcome Taxonomy

Every cell in this study resolves to exactly one of five categories. The categories are decided by the contents of the harness report rather than by judgement, so the classification is reproducible from the published logs. Table 4.4 defines them and Figure 4.5 presents the decision procedure as a state machine.

Table 4.4: Outcome Categories

Category	Definition	Decision rule
Resolved	All required tests pass and no previously passing test regresses.	Resolution flag is true.
Partial	The patch applied and made at least one required test pass, but not all.	Flag false and at least one FAIL_TO_PASS test passing.
No progress	The patch applied cleanly but no required test changed state.	Flag false, patch applied, no FAIL_TO_PASS test passing.
Regression	The patch applied and caused previously passing tests to fail.	At least one PASS_TO_PASS test failing.
Not applied	No valid patch reached the harness.	No report exists, or the patch was not applied.

Two conventions govern the use of this taxonomy. First, the regression category overlaps the partial and no-progress categories by construction, since a patch may both fail to make progress and break existing behaviour. Where this occurs the regression is reported alongside the primary category rather than replacing it, so that an outcome appears as no progress with a stated number of regressions. Second, the not-applied category is subdivided in Chapter 5 according to cause, distinguishing a malformed patch, a rejected edit, a generation refused by the provider, and a cell never attempted, because these have different methodological implications and only the first two say anything about the model's output.

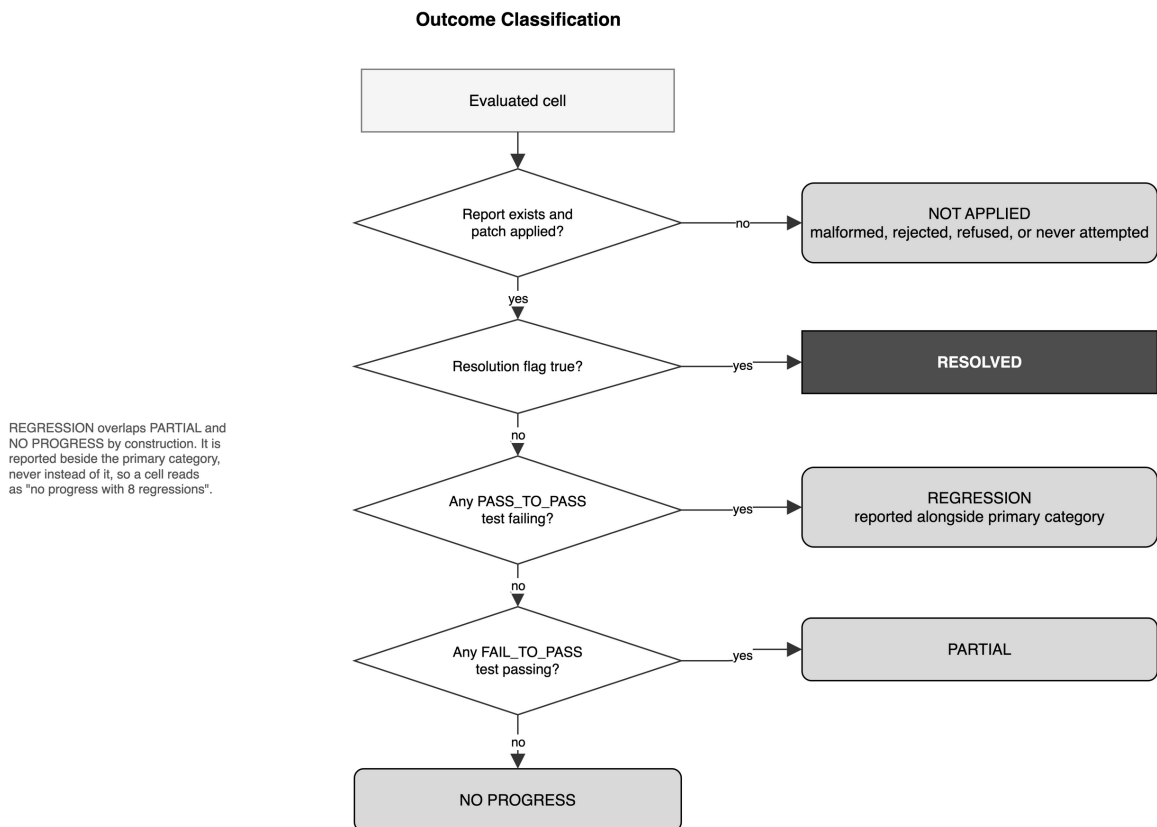


Figure 4.5 Outcome Classification State Machine

4.15 The Fairness Retrofit as an Experimental Control

Section 4.8 described the change from a model-authored unified diff to programmatic application of search-and-replace blocks. This section records why that change is treated as an experimental control rather than a bug fix, and how results obtained before and after it are handled.

Under the original representation, 4 of the 8 members of the failure pool had failed without any test being executed. A comparison of recovery strategies conducted over that pool

would have been substantially a comparison of the strategies' facility with a positional text format. The independent variable under study is the post-failure information supplied to the model, and patch formatting is a nuisance variable that was varying freely and influencing the outcome. Removing it is therefore a control, and the term fairness retrofit is used because its purpose is to ensure that the three conditions are compared on the quality of their reasoning rather than on their ability to count lines.

The retrofit was applied uniformly to all three conditions. No condition received it earlier than another, and no condition was evaluated under a mixture of the two representations within a single reported comparison. Results obtained under the original representation are reported separately in Chapter 5 and are never pooled with results obtained after the retrofit. Figure 4.6 contrasts the two pipelines.

The retrofit also has a consequence that was not anticipated when it was introduced. Because it altered how an edit is expressed while leaving the model, the prompt content, and the task unchanged, it constitutes an unplanned but clean manipulation of a single variable. The comparison between pre-retrofit and post-retrofit outcomes therefore answers a question of its own, distinct from the question the three conditions were designed to answer. Section 5.3 reports that comparison, and Section 5.9 argues that its result is the most substantial finding of the study.

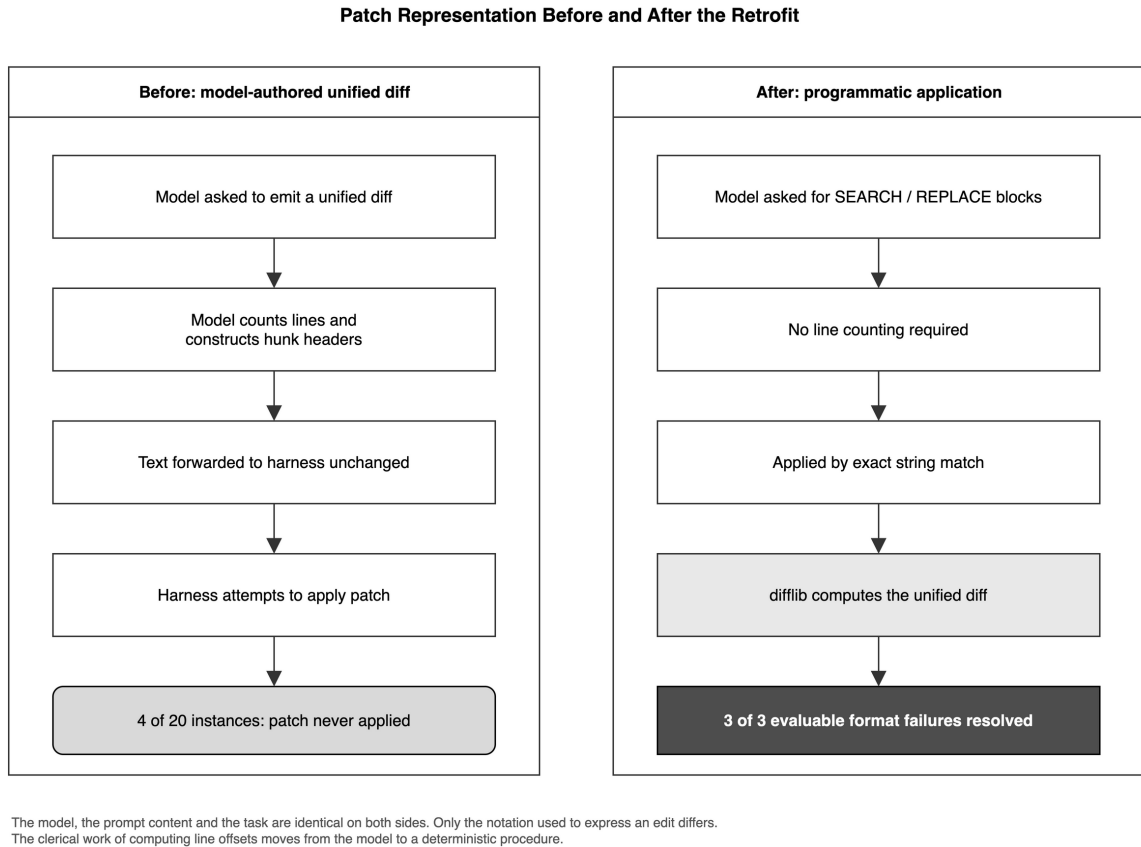


Figure 4.6 Patch Representation Before and After the Retrofit

4.16 Second Recovery Round

A single recovery attempt cannot distinguish a strategy that is ineffective from one whose effect happened not to materialise on a particular draw. Research Question 5 accordingly requires that each condition be applied more than once, and a second recovery round was conducted for this purpose.

In the second round each condition was applied again to the failure pool, with its inputs advanced by one step. Blind Retry was issued the original prompt once more, unchanged, since by definition it carries no history. Reflection-only was supplied with the response it had produced in the first round in place of the baseline response, and the prompt was amended to state that the two preceding attempts had both failed. Diagnose+Revise was supplied with evidence freshly extracted from the evaluation of its own first-round patch, and the prompt was amended to state that the previous attempt had been based on a diagnosis and had nonetheless not resolved the issue. Each condition therefore revised its own work rather than the baseline, which is the sequential behaviour the recovery literature describes.

The same truncation rule was applied to the second-round evidence as to the first, retaining the final 3,000 characters of the test output.

Coverage of the second round was incomplete, for the reasons given in Section 4.5. Of the four logic-class instances, each condition was evaluated on three in the second round, but not on the same three: the instance omitted differs between conditions according to which combinations the request ceiling permitted on the day each was run. Section 5.6 reports the coverage cell by cell. This is a genuine limitation of the robustness check and is stated as such: the study can report that outcomes were identical wherever a cell was evaluated in both rounds, and it cannot report that they were identical across the whole pool.

4.17 Reproducibility and Experimental Controls

This section records what was held constant, what was allowed to vary, and what could not be controlled.

4.17.1 Quantities Held Constant

The model, the access configuration, the benchmark, the task sample, the method of retrieving repository context, the method of identifying the target file, the patch representation within a reported comparison, the evaluation harness, the container configuration, and the shared portion of the prompt were identical across all three conditions. The ordering of tasks within a run was fixed. The number of model invocations per cell was one in every condition, so no condition benefited from additional attempts.

4.17.2 The Quantity Deliberately Varied

Only the post-failure information supplied to the model was varied, in the nested manner set out in Section 4.13.1 and Table 4.3.

4.17.3 Quantities Not Controlled

Sampling was not controlled. As recorded in Section 4.5.1, temperature, nucleus sampling, top-k, and random seed were left unspecified and the software development kit defaults were applied. Each generation is consequently a single draw from an unpinned distribution, and re-executing the pipeline may not reproduce an individual patch. The claim this study makes is that the procedure is reproducible and auditable, every script, prompt, prediction file, and evaluation log being published, and not that individual generations are deterministic. This is a genuine limitation and is repeated in Section 5.12.

The number of observations per cell was also not controlled in the sense ordinarily required for statistical inference. Each cell represents a single generation, so no variance estimate is available for any individual cell and no significance test is reported anywhere in this dissertation. Where Chapter 5 states that outcomes did not differ between conditions, the statement is descriptive of the observed cells and is not a claim of statistical equivalence. Obtaining multiple observations per cell would require a request allowance the study did not have, and is identified as the first priority for extension in Section 6.6.

Against these limitations, one observation acquires additional weight. On the instance requiring sixteen tests to pass, an identical sub-test score was returned in six separate cells spanning three conditions and two rounds. Those six figures were produced by six independent generations under unpinned sampling, not by one generation replayed six times. Agreement obtained in that manner constrains the behaviour of the system more tightly than a seeded replication would, and Section 5.7 develops the point.

4.18 Ethical and Practical Considerations

4.18.1 Provenance and Licensing of Materials

All software used in this study is publicly available and used in accordance with its licence. The benchmark instances derive from public repositories whose licences permit research use. No proprietary source code, no private repository, and no personal data of any kind was accessed. Generated patches were applied only inside disposable containers and were never contributed to any upstream project.

4.18.2 Resource Constraints and Their Disclosure

The study was conducted without funding, using a free access tier permitting 20 generation requests per day. Section 4.5.2 sets out the arithmetic: a single complete recovery round over the failure pool requires 24 requests, so full coverage within one day was not attainable. The consequence is visible in the results, where a proportion of the combinations of instance, strategy, and round were never evaluated.

This is disclosed as a methodological fact rather than confined to a limitations section, and the coverage achieved is reported cell by cell in Chapter 5, because the alternative practices are both worse. Presenting the evaluated cells without stating the denominator would imply a completeness the study does not have, and recording the unevaluated cells as failures would credit the strategies with outcomes they were never given the chance to produce. A study

conducted at zero cost is legitimately bounded by what zero cost permits, provided the bound is stated.

4.18.3 Provider Refusal and Benchmark Contamination

One instance in the failure pool, `astropy__astropy-14182`, could not be evaluated under any recovery condition. Generation was terminated by the provider with the finish reason `RECITATION`, the response given when output is judged to reproduce memorised training data too closely. The refusal recurred on every attempt and under every condition, and the instance is therefore excluded from all three conditions rather than from any one of them. No strategy is advantaged or disadvantaged by the exclusion. Its effect on the study is to reduce the pool of instances on which a recovery attempt could be evaluated from 8 to 7.

The refusal is reported here, rather than recorded silently as an exclusion, because it carries methodological information that bears directly on the validity of the benchmark. SWE-bench is constructed from public repositories whose relevant commits predate the training cut-off of contemporary models, and the possibility that reference solutions are present in model training data is the principal external threat to the validity of results obtained on it. That possibility is ordinarily a matter for conjecture. In this instance the provider's own filter declined to emit the output on the ground that it too closely reproduced memorised material, which constitutes a direct indication, from the provider rather than from the experimenter, that benchmark content is present in the training corpus.

Two consequences follow. The first is that the baseline resolution rate reported in Chapter 5 should be read as an upper bound: where a reference solution is partly recalled rather than derived, the measurement overstates repair capability. The second is that the finding is of interest independently of the recovery comparison, and Section 5.10 develops it. Systematic screening of benchmark instances for contamination is identified as future work in Section 6.6.

4.18.4 Reporting Standards Adopted

Three reporting commitments follow from the foregoing and are observed throughout Chapter 5. Unevaluated cells are marked as unevaluated and never as failures. Coverage is stated wherever a claim of consistency is made. Effectiveness is reported together with cost, since a strategy returning an identical outcome at three times the input volume is a finding that an effectiveness measure alone does not express.

4.19 Chapter Summary

This chapter has specified the experimental methodology of the study. The experiment evaluates single-shot, oracle-localized, single-file patch generation against the first 20 instances of SWE-bench Lite, using one model throughout and verifying every candidate modification by executing the project test suite inside an isolated container. The baseline stage resolved 12 instances and produced a failure pool of 8, comprising 4 instances whose patch applied but failed the required tests and 4 whose patch could not be applied at all.

Three recovery strategies were applied to that pool across two rounds. The strategies form a strictly nested sequence in which each supplies everything available to the preceding one and adds a single further element: Blind Retry supplies nothing about the failure, Reflection-only adds the model's own previous attempt together with a requirement to critique it, and Diagnose+Revise adds the observed output of the failed test run. Because the conditions are nested and every other quantity is held constant, any difference in outcome is attributable to the added information.

Three methodological decisions shape how the results are to be read. The patch representation was changed partway through the study, from a model-authored unified diff to programmatic application of search-and-replace blocks, in order to prevent patch formatting from confounding the comparison; results obtained under the two representations are reported separately. Outcomes are classified into five categories and supplemented by sub-test counts, because the benchmark's binary flag conceals partial progress. Cells that were never evaluated, whether because the daily request ceiling was reached or because an edit was rejected before any patch was produced, are marked as unevaluated rather than counted as failures.

The limitations of the design are stated where they arise rather than deferred. The task sample is a contiguous prefix of the benchmark drawn from two repositories and is not random. Sampling parameters were not fixed, so individual generations are not reproducible although the procedure is. Each cell represents a single observation, so no significance test is reported. Coverage of the comparison is incomplete and is reported cell by cell. One instance was excluded because the provider refused to generate for it, an event that is itself reported as evidence bearing on benchmark contamination.

Chapter 5 presents the results obtained under this methodology, beginning with the baseline and the classification of its failures, proceeding to the effect of the patch-representation retrofit, and then to the comparison of the three recovery strategies across both rounds.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the results of the experiment specified in Chapter 4. Sections 5.1 to 5.6 report the observations: the baseline outcome across the 20 selected instances, the classification of the resulting failures, the effect of the change in patch representation, and the outcome of each recovery strategy in each of the two rounds. These sections report what was observed and defer interpretation. Sections 5.7 to 5.13 then compare the strategies, interpret the null result, position the findings against the literature reviewed in Chapter 3, and state the threats to validity.

All figures reported in this chapter are derived by script from the structured reports written by the evaluation harness, and the derivation can be re-executed against the published logs. Sub-test counts are read from the per-test status fields of those reports rather than from the summary resolution flag, for the reasons given in Section 4.14.1. Cells that were never evaluated are marked as such throughout and are never presented as failures, in accordance with the reporting commitments stated in Section 4.18.4.

5.1 Baseline Results

Each of the 20 selected instances received one generation attempt under the conditions described in Section 4.10. Of these, 12 instances were resolved, meaning that every required test passed and no previously passing test regressed. This corresponds to a baseline resolution rate of 60 per cent. The remaining 8 instances were not resolved and constitute the failure pool.

Table 5.1 records the outcome for every instance, together with the number of FAIL_TO_PASS tests that passed out of the number required and the number of PASS_TO_PASS tests that regressed. Figure 5.1 presents the same distribution graphically.

Table 5.1: Baseline Results, All 20 Instances

Instance	Outcome category	FAIL_TO_PAS S	Regressions
astropy__astropy-6938	Resolved	2 of 2	0
astropy__astropy-12907	Resolved	2 of 2	0
astropy__astropy-14365	Resolved	1 of 1	0
astropy__astropy-14995	Resolved	1 of 1	0

Instance	Outcome category	FAIL_TO_PAS S	Regressions
django__django-10914	Resolved	1 of 1	0
django__django-10924	Resolved	1 of 1	0
django__django-11001	Resolved	2 of 2	0
django__django-11049	Resolved	1 of 1	0
django__django-11099	Resolved	3 of 3	0
django__django-11133	Resolved	1 of 1	0
django__django-11179	Resolved	1 of 1	0
django__django-11422	Resolved	1 of 1	0
astropy__astropy-7746	No progress	0 of 1	0
django__django-11019	No progress, with regressions	0 of 16	33
django__django-11283	No progress	0 of 1	0
django__django-11564	No progress	0 of 2	0
astropy__astropy-14182	Not applied (malformed patch)	Not run	Not run
django__django-11039	Not applied (malformed patch)	Not run	Not run
django__django-11583	Not applied (malformed patch)	Not run	Not run
django__django-11620	Not applied (malformed patch)	Not run	Not run

Two observations are recorded here without comment and returned to later. The instance django__django-11019 is the only one in the sample requiring a large number of tests to pass, 16 in total, and is also the only instance at which the baseline patch caused regressions, breaking 33 previously passing tests. All other instances in the sample require between one and three tests to pass.

Baseline Outcome Breakdown, 20 Instances



Outcome	Count	Share
Resolved on first attempt	12 of 20	60 per cent
Logic-class failure patch applied, required tests failed	4 of 20	20 per cent
Format-class failure patch could not be applied	4 of 20	20 per cent
Failure pool	8 of 20	40 per cent

The two failure classes are equal in size. Half the failure pool entered it for a reason unconnected with software reasoning.

Figure 5.1 Baseline Outcome Breakdown

5.2 Baseline Failure Classification

The 8 unresolved instances did not fail in the same manner. Applying the taxonomy defined in Section 4.14.3 divides them into two groups of equal size, and the division is determined by a single question: whether the generated patch was successfully applied before any test was executed.

5.2.1 Logic-Class Failures

Four instances produced a patch that applied cleanly to the repository. The evaluation therefore proceeded, the designated tests were executed, and the required tests did not pass. These are `astropy__astropy-7746`, `django__django-11019`, `django__django-11283`, and `django__django-11564`. For these instances the evaluation yields information about the correctness of the proposed modification, because the modification was actually made and its behavioural consequences observed. They are referred to throughout this chapter as logic-class failures.

5.2.2 Format-Class Failures

Four instances produced a patch that could not be applied at all. In one case the harness reported that only unusable content was present in the patch input. For these instances no test was executed, so the evaluation yields no information whatsoever about whether the intended modification was correct; it establishes only that the modification was not expressible in the requested format. These are `astropy__astropy-14182`, `django__django-11039`, `django__django-11583`, and `django__django-11620`. They are referred to as format-class failures.

5.2.3 Consequence for the Comparison

That the failure pool divides evenly between the two classes is the central methodological observation of this study. Half of the pool entered it for a reason unrelated to software reasoning. A comparison of recovery strategies conducted over the pool as a whole, reporting a single aggregate recovery rate, would therefore have measured two different things at once and reported their sum. Every results table from this point forward distinguishes the two classes, and Section 5.9 sets out why the distinction changes the conclusion the study reaches.

5.3 The Patch-Formatting Confound

The format-class failures were initially treated as ordinary failures. The recovery strategies were applied to the full pool of 8 instances using the original patch representation, in which the model was asked to emit a unified diff directly. Inspection of the resulting logs showed that the failures of this group were not distributed across the varied causes that a reasoning failure would produce, but were concentrated in the mechanics of the diff format itself: hunk headers declaring line positions that did not correspond to the target file, and in one case output that the patch utility could not parse at all.

This prompted the change of representation described in Section 4.8.2 and treated as an experimental control in Section 4.15. Under the revised arrangement the model expresses each edit as a pair of literal text blocks, the edit is applied programmatically by exact string match, and the unified diff required by the harness is computed mechanically from the original and modified files. The model is no longer required to count lines or construct hunk headers.

The effect was immediate and is reported here as an observation. Of the 4 format-class instances, 3 were subsequently resolved. The fourth, `astropy__astropy-14182`, could not be evaluated at all for the reason recorded in Section 4.18.3, the provider having declined to

generate output for it. Every format-class instance that could be evaluated after the change of representation was resolved.

The instances `django__django-11039` and `django__django-11583` were resolved under Blind Retry while the original representation was still in use, and were not subsequently re-evaluated under the remaining two conditions, the requests being reserved for the logic-class instances for the reason given in Section 4.5.2. The instance `django__django-11620` was evaluated under all three conditions after the retrofit and was resolved by each of them.

A second effect of the change of representation was observed on a logic-class instance and is reported in Section 5.4, because it concerns the sub-test counts rather than the resolution flag.

5.4 Results After the Fairness Retrofit

The change of patch representation altered the measured outcome on `django__django-11019`, the instance requiring 16 tests to pass. This instance is not recorded as resolved either before or after the change, so the alteration is invisible to the benchmark's resolution flag and is observable only in the sub-test counts introduced for this purpose in Section 4.14.1.

Under the original representation the baseline patch for this instance passed 0 of the 16 required tests and caused 33 previously passing tests to fail. Under the revised representation the patch passed 6 of the 16 required tests and caused no regressions. The change is therefore from 0 to 6 on the required tests, and from 33 to 0 on the regressions. Figure 5.2 presents both movements.

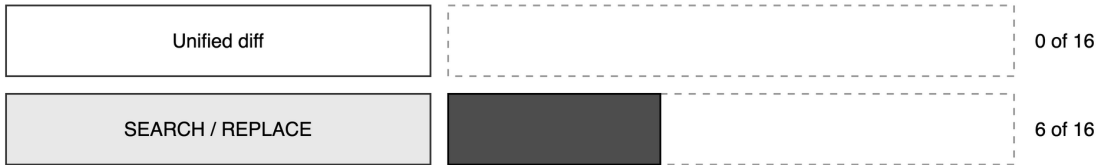
This is the largest single change in measured behaviour observed anywhere in this study. It was produced without altering the model, the sampling configuration, the issue description, the repository context, or the instructions governing the reasoning required. The only quantity that changed was the format in which the edit was expressed. Section 5.9 examines what follows from this.

The same figure of 6 of 16 was subsequently returned by every evaluated recovery cell for this instance, under all three strategies and in both rounds. The tables in Sections 5.5 and 5.6 record this, and Section 5.7 examines it.

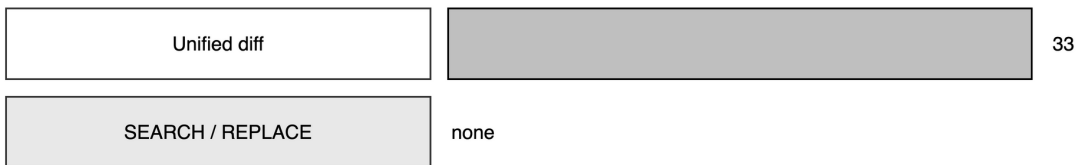
Sub-Test Recovery on django__django-11019

The benchmark records this instance as UNRESOLVED both before and after. Only the sub-test counts reveal the change.

FAIL_TO_PASS tests passing (dashed track = all 16 required)



PASS_TO_PASS tests broken (fewer is better)



Every evaluated recovery cell then returned the identical score		
Blind Retry R1 6 of 16	Blind Retry R2 6 of 16	Reflection R1 6 of 16
Reflection R2 6 of 16	Diagnose R1 6 of 16	Diagnose R2 6 of 16

Six independent generations, three strategies, two rounds, sampling not pinned by temperature or seed.
The retrofit moved this instance. Nothing told to the model about its failure moved it again.

Figure 5.2 Sub-Test Recovery on django__django-11019

5.5 Recovery Outcomes: Round One

Table 5.2 reports the outcome of each recovery strategy applied to each instance in the failure pool during the first recovery round. Cells in which no evaluation was performed are marked as not evaluated. As set out in Sections 4.5.2 and 4.8.2, such a cell arises either because the daily request ceiling was reached before the combination could be attempted, or because the model's proposed edit was rejected by the applicator and no patch was produced. In neither case does the cell constitute evidence about the strategy, and it is not counted as a failure.

Table 5.2: Round One Recovery Outcomes

Instance	Class	Blind Retry	Reflection-only	Diagnose+Revise
astropy__astropy-7746	Logic	Not evaluated	Not evaluated	No progress, 0 of 1
django__django-11019	Logic	Partial, 6 of 16	Partial, 6 of 16	Partial, 6 of 16
django__django-11283	Logic	Not evaluated	Not evaluated	No progress, 0 of 1, with 8 regressions
django__django-11564	Logic	No progress, 0 of 2	Not evaluated	No progress, 0 of 2
astropy__astropy-14182	Format	Excluded	Excluded	Excluded
django__django-11039	Format	Resolved (pre-retrofit)	Not evaluated	Not evaluated
django__django-11583	Format	Resolved (pre-retrofit)	Not evaluated	Not evaluated
django__django-11620	Format	Resolved	Resolved	Resolved

Considering first the four logic-class instances, no instance was resolved by any strategy. One instance, django__django-11019, returned a partial outcome of 6 of 16 required tests under each of the three strategies. The remaining evaluated cells returned no progress. One cell, django__django-11283 under Diagnose+Revise, additionally caused 8 previously passing tests to fail.

Coverage of the logic-class instances in this round was uneven. Diagnose+Revise was evaluated on all 4; Blind Retry on 2; Reflection-only on 1. The unevenness is a consequence of the constraint documented in Section 4.5.2 and not of any selection decision, and it means that the round-one figures alone do not support a comparison between the three strategies on equal terms. The second round, reported in Section 5.6, was conducted in part to address this.

Considering the format-class instances, the 3 that could be evaluated were resolved. The instance evaluated under all three conditions, django__django-11620, was resolved by each of them.

5.6 Recovery Outcomes: Round Two

The second recovery round was applied to the four logic-class instances under the arrangement described in Section 4.16, each strategy revising its own previous work rather than the baseline. The format-class instances were not re-evaluated in this round, the three evaluable members of that group having already been resolved. Table 5.3 reports the outcome.

Table 5.3: Round Two Recovery Outcomes

Instance	Blind Retry	Reflection-only	Diagnose+Revise
astropy__astropy-7746	No progress, 0 of 1	No progress, 0 of 1	No progress, 0 of 1
django__django-11019	Partial, 6 of 16	Partial, 6 of 16	Partial, 6 of 16
django__django-11283	No progress, 0 of 1	No progress, 0 of 1	Not evaluated
django__django-11564	Not evaluated	Not evaluated	No progress, 0 of 2

No instance was resolved by any strategy in the second round. Every evaluated cell returned the same outcome category as the corresponding cell in the first round, and where sub-test counts are available they are identical. In particular `django__django-11019` returned 6 of 16 under every strategy in both rounds, and `astropy__astropy-7746` returned 0 of 1 under every strategy in both rounds. No regressions were observed in any second-round cell, including the cell that had produced 8 regressions in the first round.

Coverage in this round was more even than in the first but remained incomplete. Each strategy was evaluated on 3 of the 4 logic-class instances, but not on the same 3: Blind Retry and Reflection-only were evaluated on `astropy__astropy-7746`, `django__django-11019` and `django__django-11283`, whereas Diagnose+Revise was evaluated on `astropy__astropy-7746`, `django__django-11019` and `django__django-11564`.

5.6.1 Coverage Across Both Rounds

Because coverage is incomplete in both rounds and incomplete in different places, the extent of the evidence supporting the comparison is stated explicitly. The comparison of three strategies over four logic-class instances across two rounds defines 24 cells. Of these, 16 were evaluated and 8 were not. Table 5.4 gives the distribution.

Table 5.4: Coverage of the Logic-Class Comparison

Strategy	Round one	Round two	Total evaluated
Blind Retry	2 of 4	3 of 4	5 of 8
Reflection-only	1 of 4	3 of 4	4 of 8
Diagnose+Revise	4 of 4	3 of 4	7 of 8
All strategies	7 of 12	9 of 12	16 of 24

The claim that the observations support is therefore consistency across every cell that was evaluated, in both rounds and under all three strategies, together with the coverage figure of 16 cells of 24. The observations do not support a claim of consistency across the failure pool as a whole, and no such claim is made in this dissertation. Section 5.12 records the incompleteness of coverage among the threats to validity.

Sections 5.7 to 5.13 now interpret these observations, beginning with the comparison between the three strategies and the cost at which each was obtained.

5.7 Cross-Strategy Comparison

This section compares the three recovery strategies on the logic-class instances, which are the instances from which any inference about software reasoning can be drawn. The format-class instances are treated separately in Section 5.9, because what they demonstrate is of a different kind.

5.7.1 Outcome Comparison

Across the four logic-class instances, in two rounds, no instance was resolved by any of the three strategies. The measured outcome is identical in every cell where two or more strategies were evaluated on the same instance in the same round. Table 5.5 sets out the sub-test scores, which permit a finer comparison than the resolution flag and reveal no separation between the strategies either.

Table 5.5: Sub-Test Scores by Condition, Logic-Class Instances

Condition and round	astropy-7746	django-11019	django-11283	django-11564
Baseline	0 of 1	0 of 16, 33 regressions	0 of 1	0 of 2
Blind Retry, round one	Not evaluated	6 of 16	Not evaluated	0 of 2
Blind Retry, round two	0 of 1	6 of 16	0 of 1	Not evaluated
Reflection-only, round one	Not evaluated	6 of 16	Not evaluated	Not evaluated
Reflection-only, round two	0 of 1	6 of 16	0 of 1	Not evaluated
Diagnose+Revise, round one	0 of 1	6 of 16	0 of 1, 8 regressions	0 of 2
Diagnose+Revise, round two	0 of 1	6 of 16	Not evaluated	0 of 2

The instance `django__django-11019` is the most informative, being the only instance evaluated in all six recovery cells and the only one requiring a substantial number of tests to

pass. It returned 6 of 16 in every one of those six cells. The identity holds across strategies that received quite different information: one that was told nothing about the failure, one that received its own previous attempt and a requirement to criticise it, and one that additionally received the observed output of the failing tests. Section 5.8 considers what this implies.

Figure 5.3 presents the complete comparison matrix, with the cells that were not evaluated shown as such rather than omitted or inferred.

Cross-Strategy Comparison Matrix, Logic-Class Instances

Instance	Blind Retry		Reflection-only		Diagnose+Revise	
	Round 1	Round 2	Round 1	Round 2	Round 1	Round 2
astropy-7746	not evaluated	0 of 1	not evaluated	0 of 1	0 of 1	0 of 1
django-11019	6 of 16	6 of 16	6 of 16	6 of 16	6 of 16	6 of 16
django-11283	not evaluated	0 of 1	not evaluated	0 of 1	0 of 1*	not evaluated
django-11564	0 of 2	not evaluated	not evaluated	not evaluated	0 of 2	0 of 2

* Diagnose+Revise additionally broke 8 PASS_TO_PASS tests on this cell. Blind Retry broke none.

Legend

evaluated

not evaluated

No instance was resolved by any strategy in either round. 16 of the 24 cells were evaluated; 8 were not. Unevaluated cells are shown as such and are never counted as failures.

Figure 5.3 Cross-Strategy Comparison Matrix

5.7.2 The One Case Where the Strategies Differed

In exactly one cell did a strategy produce an outcome that another did not, and the difference was unfavourable to the better-informed strategy. On `django__django-11283` in the first round, Diagnose+Revise produced a patch that applied cleanly, failed the single required test, and additionally caused 8 previously passing tests to fail. Blind Retry, evaluated on the same instance in the second round, produced no regressions, as did Reflection-only.

This observation is recorded rather than averaged away, because averaging it into an aggregate recovery rate would remove the only point of separation the data contains. Its interpretation must be cautious: it is a single cell, produced by a single unpinned generation, and it may be a sampling artefact. It is nonetheless consistent with a mechanism worth stating. The Diagnose+Revise prompt directs the model to a specific root cause grounded in the evidence supplied, and that evidence is the final portion of a test log. A directive to address the proximate cause of a specific observed failure invites a narrowly targeted edit, and a narrowly targeted edit in a framework as interdependent as Django can satisfy the immediate complaint while disturbing behaviour elsewhere. On this instance the additional information did not merely fail to help; it accompanied a more damaging modification.

5.7.3 Cost of Each Strategy

Effectiveness is reported here together with the cost at which it was obtained, in accordance with Section 4.18.4. The three strategies do not cost the same. Because the complete target file is retransmitted on every request, and because each condition adds further material to the prompt without removing anything, input volume rises across the sequence: Reflection-only carries approximately twice the input of Blind Retry, and Diagnose+Revise approximately three times. For the largest instance in the sample the target file alone accounts for some 31,100 tokens on every request, and no caching was employed. Each request additionally incurred the 60-second delay described in Section 4.5.2.

Taken together with the outcome comparison, the result is therefore not simply that the additional information failed to improve recovery. It is that a strategy costing three times as much in input volume, and requiring the additional infrastructure to capture, store, and truncate execution evidence, returned an outcome identical to that of re-issuing the original prompt unchanged. Expressed in the terms the recovery literature has begun to adopt, the verified recovery rate on logic-class failures was zero for all three strategies, while the recovery effort differed by a factor of three. Figure 5.4 presents cost against outcome.

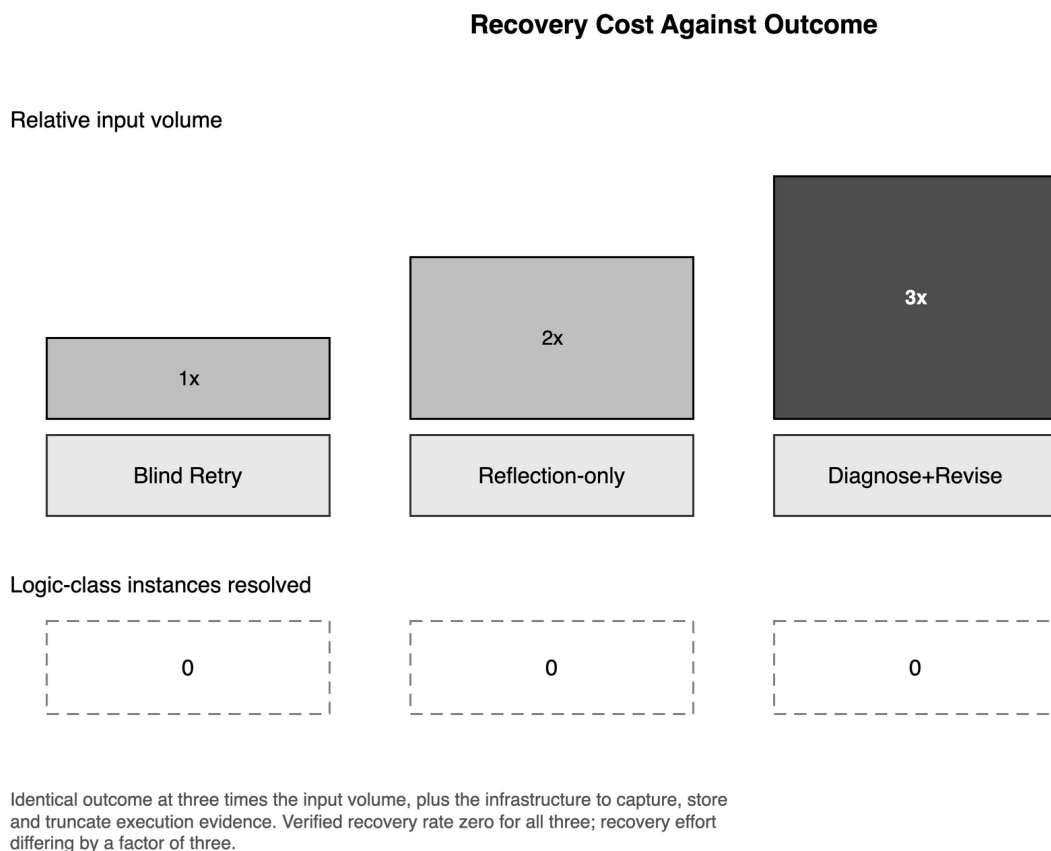


Figure 5.4 Recovery Cost by Strategy

5.8 Interpretation of the Null Result

No recovery strategy resolved any logic-class failure, and the better-informed strategies did not outperform the uninformed one. Three explanations are compatible with this observation, and they are not equally well supported.

5.8.1 Explanation One: The Information Was Not Available

The first explanation is that the model lacked what it needed. This explanation is the weakest, and the design of the experiment is what weakens it. The target file was supplied in full rather than discovered, so the model cannot have examined the wrong file. The issue description was supplied verbatim. In the Reflection-only condition the model's own previous reasoning was returned to it in the form in which it had been expressed. In the Diagnose+Revise condition the observed output of the failing tests was supplied and explicitly labelled as ground truth. The information required to identify the defect was, by construction, present in the prompt. As argued in Section 4.7.2, oracle localization makes this explanation harder to sustain rather than easier.

One qualification applies. The execution evidence was truncated to its final 3,000 characters, which for the largest log retained approximately 0.2 per cent of the output. The tail of a Python test log carries the exception and the failing assertion, so the most diagnostic portion is likely to have been retained; but in a run where several tests fail, earlier failures were discarded. The Diagnose+Revise condition was therefore tested with a deliberately narrow evidence window, and this study cannot exclude the possibility that a wider window would have produced a different outcome. This is recorded as a limitation of the condition as implemented and as a specific target for future work in Section 6.6.

5.8.2 Explanation Two: The Tasks Were Not Solvable

The second explanation is that the four logic-class instances could not have been resolved by any strategy, in which case no strategy could demonstrate an advantage and the comparison is uninformative rather than negative.

This explanation cannot be excluded. The human screening exercise undertaken during the construction of SWE-bench Verified established that a substantial proportion of SWE-bench instances are problematic as stated, the two principal causes being issue descriptions that underspecify the required behaviour and tests that impose requirements not derivable from the issue text. No independent solvability audit of the four instances in this study's pool was

conducted, for the reason given in Section 4.4.3. One instance, `django__django-11283`, is known from its test output to fail because the test invokes the modified function with an argument the issue description does not mention, which is characteristic of the second category. It is therefore possible that part of the null result reflects the benchmark rather than the strategies. This is the most serious limitation of the empirical component of this dissertation and is recorded as such in Section 5.12.

5.8.3 Explanation Three: The Information Was Available and Not Used

The third explanation is that the additional information was present and did not alter what the model produced. The pattern on `django__django-11019` bears on this directly and is the strongest positive evidence the study offers.

That instance returned 6 of 16 in six separate cells. Those six figures were produced by six independent generations, spanning three strategies and two rounds, under sampling that was not pinned by a fixed temperature or seed, as recorded in Section 4.17.3. Six unpinned draws converging on precisely the same partial solution is not the signature of a system responding to its inputs and being deflected by noise. It is the signature of a system arriving at the same place irrespective of what it is told about having been there before. The model appears to reach a stable partial understanding of the issue, one sufficient to satisfy 6 of the 16 requirements, and neither self-criticism nor the observed test output moved it from that position.

The absence of a fixed seed, which Section 4.17.3 records as a limitation for reproducibility, therefore strengthens this particular inference. Had the generations been seeded, identical outputs would have been unremarkable and would have demonstrated nothing about the influence of the prompt. Obtained from unpinned draws, the agreement constrains the system's behaviour more tightly than a deterministic replication could.

5.8.4 Assessment

The first explanation is largely excluded by the design, subject to the qualification about the evidence window. The second cannot be excluded and requires a solvability audit that this study did not perform. The third is positively supported on the one instance where the data are rich enough to test it. The defensible conclusion is therefore narrower than a general claim about self-reflection: on the logic-class failures evaluated here, supplying a coding system with its own failed attempt, or with the observed output of the tests it failed, did not change what it produced, and on the instance affording the closest examination the system returned the identical partial solution under every condition. Whether this generalises beyond four

instances, two repositories, and one model is not established by this study, and Section 5.12 states so.

5.9 Patch Reliability and Reasoning Correctness

The comparison between the two failure classes yields the clearest result of this study, and it concerns Research Question 4.

Of the four format-class failures, three could be evaluated after the change of patch representation and all three were resolved. Of the four logic-class failures, none was resolved by any strategy in either round. Stated as rates, recovery on evaluable format-class failures was 3 of 3, and recovery on logic-class failures was 0 of 4. The two classes differ in the mechanism by which they entered the failure pool, and they responded to entirely different interventions.

The intervention that resolved the format-class failures altered nothing about the model's reasoning. It did not supply additional information, did not request reflection, and did not provide execution evidence. It changed the notation in which the edit was expressed, moving the clerical work of computing line offsets from the model to a deterministic procedure. The evidence on `django__django-11019` reported in Section 5.4 makes the same point within a single instance: the same change of notation moved that instance from 0 of 16 required tests with 33 regressions to 6 of 16 with none, while the model, the prompt content, and the task remained unchanged.

The finding may therefore be stated as follows. Across this experiment, changing how the edit was expressed altered outcomes substantially. Changing what the model was told about its own failure did not alter them at all. Both interventions were applied to the same instances, by the same pipeline, under the same evaluation.

5.9.1 Consequence for the Evaluation of Coding Agents

This has a direct implication for how recovery mechanisms are evaluated, and it is the principal methodological contribution of this dissertation. An aggregate resolution rate cannot distinguish the two mechanisms. Had this study reported a single recovery figure over the undivided pool of 8 instances, the result would have been 3 recoveries of 7 evaluable instances, approximately 43 per cent, and that figure would have appeared to demonstrate that the recovery strategies worked. It would have been arithmetically correct and substantively misleading, because every one of the three recoveries was obtained on an instance whose

original failure was a malformed patch, and none on an instance whose original failure was a defect of reasoning.

It follows that a study reporting an improved resolution rate after the introduction of a reflection or diagnosis mechanism cannot be assumed to have demonstrated improved reasoning, unless it also reports how many of its prior failures were patch application failures. Where a system's patch-application reliability and its reasoning correctness are not reported separately, a gain in the former is indistinguishable from a gain in the latter. Section 5.10 states this as a general reporting recommendation, and Section 3.10.4 identified the absence of this separation in the existing literature as one of the gaps motivating the study.

5.10 Methodological Findings

Five findings of this study concern how recovery in coding systems should be measured rather than how it behaves. They constitute the response to Objective 8.

5.10.1 Patch Application Must Be Reported Separately

For the reasons set out in Section 5.9, the proportion of attempts whose patch applied should be reported alongside, and separately from, the proportion that resolved the task. In this study half the failure pool consisted of patches that never applied, and no conclusion about reasoning could be drawn from them until the representation was repaired.

5.10.2 Binary Resolution Rates Conceal the Largest Effects

The most substantial change observed in this study, the movement on `django__django-11019` from 0 of 16 required tests with 33 regressions to 6 of 16 with none, is invisible to the benchmark's resolution flag, which records the instance as unresolved in both states. Reporting sub-test counts is inexpensive, the data being present in the harness output already, and in this case it was the difference between observing the study's principal effect and missing it entirely.

5.10.3 A Resampling Control Is Necessary

Any claim that a reflection or diagnosis mechanism improves recovery requires comparison against simply attempting the task again. In this study the uninformed control matched both better-informed strategies on every evaluated logic-class cell. Without that control, the partial progress observed after the retrofit could have been attributed to reflection, since it coincided with it in time.

5.10.4 Effectiveness Must Be Reported With Cost

Diagnose+Revise consumed approximately three times the input volume of Blind Retry and required additional infrastructure to collect and truncate execution evidence, for an identical outcome. An effectiveness measure reported alone would not express this. The pairing of a recovery rate with the effort expended in obtaining it, as Section 5.7.3 reports, is necessary for the comparison to be meaningful to anyone deciding whether to deploy such a mechanism.

5.10.5 Unevaluated Cells Must Not Be Recorded as Failures

Of the 24 cells defined by the logic-class comparison, 8 were never evaluated. Had these been recorded as failures, the strategies would have been charged with outcomes they were never given the opportunity to produce, and the apparent coverage of the study would have been overstated. Reporting coverage explicitly, as Table 5.4 does, is the minimum requirement for a comparison conducted under resource constraints.

5.10.6 Provider Refusals Are a Contamination Signal

The exclusion of `astropy__astropy-14182` recorded in Section 4.18.3 arose because the provider declined to generate output on the ground that it reproduced memorised training data too closely. Benchmark contamination is ordinarily inferred indirectly, by comparing performance across instances of differing vintage or by probing for memorisation. Here it was signalled by the provider's own filter, independently of the experimenter. Such refusals are recorded in generation logs and are straightforward to detect; treating them as contamination evidence rather than as an inconvenience to be excluded silently costs nothing and yields information that bears on the validity of every result obtained on the benchmark.

5.11 Comparison With the Existing Literature

The findings of this study stand in a definite relation to the literature reviewed in Chapter 3, agreeing with one part of it and qualifying another.

5.11.1 Agreement With Analyses of Intrinsic Self-Correction

The result obtained under the Reflection-only condition agrees with the conclusion of Huang et al. [36], who report that large language models do not reliably correct their own reasoning when the only additional input is a request to reconsider, and that apparent gains from self-correction frequently disappear once a comparable resampling control is introduced. The present study reaches the same conclusion in a setting their work did not examine: repository-level program repair, on production source files, verified by the projects' own test suites. The agreement is therefore a replication in a new domain rather than a restatement, and

it strengthens both results. The related analysis by Kamoi et al. [37] of the conditions under which self-correction succeeds is consistent with this: they identify the availability of reliable external feedback as the decisive factor, and the Reflection-only condition supplies none.

The Diagnose+Revise condition does supply external feedback, in the form of the observed test output, and on that account might have been expected to succeed where Reflection-only did not. It did not. Two candidate reasons are available and this study cannot separate them: the feedback was truncated to a narrow window, as Section 5.8.1 records; or the defects in question were not ones the model could repair given any amount of feedback, as Section 5.8.2 allows. This is the most significant point at which the present findings depart from what the literature would predict, and resolving it is the most valuable extension of the work.

5.11.2 Qualification of Reported Gains From Iterative Refinement

The Reflexion and Self-Refine frameworks [7], [8] reviewed in Sections 3.3.1 and 3.3.2 report substantial benefits from iterative self-revision. The present results do not contradict those reports, but they bound them. Those frameworks were evaluated predominantly on tasks where the correctness signal is cheap, immediate, and complete, such as unit-test-level code generation or constrained reasoning problems. Repository-level repair differs in each respect: the signal is expensive to obtain, requires container construction and full test execution, and arrives as a large log that must be truncated before it can be used. The present study suggests that the benefit of iterative refinement does not transfer automatically to this setting, and that the properties of the feedback channel, rather than the presence of a reflection step, may be what determines whether it does.

5.11.3 Consistency With Partial-Credit Critiques of Resolve Rate

Recent work criticising aggregate resolution rates argues that a binary measure discards the greater part of what an evaluation observes and that partial progress should be reported. The observation on `django__django-11019`, where the resolution flag is unchanged while the underlying sub-test counts move from 0 of 16 with 33 regressions to 6 of 16 with none, is a direct instance of that argument. The present study did not set out to test it and provides incidental but unambiguous support for it.

5.11.4 Comparison With Independently Reported Recovery Evaluations

Independent evaluation of recovery mechanisms on site-reliability and operations benchmarks provides a useful point of comparison. One such evaluation reports an

improvement from 20 of 50 tasks to 24 of 50 upon the introduction of a structured recovery mechanism, and reports that improvement as not statistically significant at that sample size. The relation to the present study is twofold. The direction of the reported effect is favourable where this study observed none, which is a genuine difference to be accounted for by domain, mechanism, or scale. But the magnitude is of an order that a sample of four instances could not have detected, which places the present null result in proper perspective: it is evidence that the effect is not large on these instances, and not evidence that no effect exists.

5.12 Threats to Validity

The limitations of this study are material and are stated in full. Table 5.6 summarises them; the subsections that follow set out the four most consequential.

Table 5.6: Threats to Validity

Threat	Type	Effect on the conclusions	Status
Task solvability not audited	Conclusion	Null result may reflect the benchmark rather than the strategies	Not mitigated; audit required
Four logic-class instances only	External	No claim of generality; effects smaller than large are undetectable	Stated; larger pool required
Sample is a contiguous prefix, two repositories	External	Not representative of SWE-bench Lite	Disclosed in Section 4.4
Coverage of 16 cells of 24	Internal	Consistency claim limited to evaluated cells	Reported cell by cell, Table 5.4
Sampling parameters not fixed	Internal	Individual generations not reproducible	Disclosed; strengthens Section 5.8.3
One observation per cell	Conclusion	No variance estimate; no significance test reported	Stated in Section 4.17.3
Single model and provider	External	Findings conditional on one model family	Stated; replication required
Benchmark contamination	External	Baseline resolution rate is an upper bound	Evidenced in Section 4.18.3
Evidence truncated to 3,000 characters	Construct	Diagnose+Revise tested with a narrow feedback window	Disclosed in Section 4.13
Oracle file localization	Construct	Rates not comparable with end-to-end systems	Disclosed in Section 4.7
Patch representation changed mid-study	Internal	Pre- and post-retrofit results not comparable	Reported separately throughout

5.12.1 Task Solvability

The most serious threat is that the logic-class instances may not have been solvable from the information supplied. A strategy cannot be shown ineffective on a task that no strategy could complete. As Section 5.8.2 records, the screening undertaken for SWE-bench Verified established that a substantial proportion of SWE-bench instances are underspecified or unfairly tested, and at least one instance in this study's pool exhibits the characteristic pattern of the latter. Until the four instances are audited against their issue descriptions and reference solutions, the null result cannot be attributed with confidence to the strategies rather than to the tasks. This audit is the first item of future work identified in Section 6.6.

5.12.2 Scale and Statistical Power

The logic-class comparison rests on four instances and one observation per cell. No variance estimate is available and no significance test is reported anywhere in this dissertation. The study can state that no effect was observed; it cannot state that no effect exists. As Section 5.11.4 notes, an effect of the magnitude reported elsewhere in the literature would not have been detectable at this scale. Claims in this chapter are accordingly confined to the observed cells.

5.12.3 Generality

The sample is a contiguous prefix of one benchmark, drawn from two repositories, with 14 of the 20 instances originating from a single project. One model from one provider was used throughout. The findings are therefore conditional on that model, those repositories, and the particular prompt formulations reported in Sections 4.11 to 4.13. Whether they hold for a different model family, for repositories with different test conventions, or under prompts differently phrased, is not established here.

5.12.4 Benchmark Contamination

As recorded in Sections 4.18.3 and 5.10.6, the provider declined to generate for one instance on the ground that the output reproduced memorised training data. This is direct evidence that benchmark content is present in the model's training corpus. Its consequence for this chapter is that the baseline resolution rate of 60 per cent should be read as an upper bound on repair capability, since an instance whose reference solution is partly recalled rather than derived inflates the measure. It does not affect the comparison between recovery strategies, all three of which operated on the same instances under the same conditions.

5.13 Chapter Summary

This chapter has reported and interpreted the results of the experiment specified in Chapter 4. At baseline, 12 of 20 instances were resolved, a rate of 60 per cent, leaving a failure pool of 8. That pool divided evenly into 4 instances whose patch applied but whose required tests did not pass, and 4 whose patch could not be applied at all.

On the logic-class instances, no strategy resolved any instance in either round. Where cells were evaluated the outcomes were identical between strategies, including at the level of sub-test counts: `django__django-11019` returned 6 of 16 required tests in all six of its recovery cells. In the single cell where the strategies diverged, the divergence was unfavourable to the best-informed strategy, Diagnose+Revise having caused 8 regressions where the other two caused none. Diagnose+Revise consumed approximately three times the input volume of Blind Retry for these identical outcomes.

On the format-class instances, all 3 that could be evaluated were resolved, following a change in the notation used to express an edit that left the model, the prompt content, and the task unaltered. The same change moved `django__django-11019` from 0 of 16 required tests with 33 regressions to 6 of 16 with none. Across this experiment, changing how an edit was expressed altered outcomes substantially, and changing what the model was told about its own failure did not alter them at all.

The principal methodological consequence is that patch-application reliability and reasoning correctness must be measured and reported separately. Had this study reported a single undivided recovery rate, it would have recorded 3 recoveries of 7 evaluable instances and appeared to demonstrate that the recovery strategies were effective, when every recovery was obtained on an instance whose original failure was mechanical.

These conclusions are bounded. Four logic-class instances, one observation per cell, coverage of 16 cells of 24, a non-random sample from two repositories, a single model, unpinned sampling, a truncated feedback window, and an unaudited assumption of task solvability all limit what may be inferred. The study reports that no effect was observed on these instances under these conditions; it does not report that no effect exists. Chapter 6 states the conclusions against each research question and sets out the work that would be required to establish more.

CHAPTER 6

CONCLUSIONS AND SUMMARY

6.1 Summary of Work Carried Out

This dissertation began with a systematic review of 35 studies published between 2023 and 2026 on self-reflection, memory, planning, and failure recovery in agentic AI coding systems [1]–[35]. The review identified a specific and validated gap: the absence of empirical evidence on whether execution-grounded, diagnosis-based recovery improves outcomes for repository-level coding systems by comparison with simpler retry or reflection-based strategies.

Building on that gap, a fully scripted and auditable empirical pipeline was designed and implemented, encompassing benchmark and task selection, retrieval of real repository context at the recorded base commit, a patch mechanism based on search-and-replace blocks with programmatically generated unified diffs, and a containerised evaluation harness executed through the Windows Subsystem for Linux. A controlled three-condition comparison of Blind Retry, Reflection-only, and Diagnose+Revise was conducted on a 20-instance pilot sample from SWE-bench Lite [19], followed by an independent second recovery round to test robustness.

The comparison was conducted after a change in the representation used to express code edits, adopted because the original representation had caused half of the failure pool to fail for reasons unconnected with software reasoning. That change is treated throughout as an experimental control, and it proved to have consequences of its own that are among the principal findings reported below.

6.2 Conclusions Against the Research Questions

The five research questions stated in Section 2.3 are answered as follows.

6.2.1 Research Question 1

What failure-recovery mechanisms have been investigated in existing research on agentic AI coding systems? The systematic review presented in Chapter 3 identified and classified the mechanisms reported across 35 studies, spanning verbal self-reflection and iterative self-revision [7]–[10], experiential and structured memory [11], [14]–[17], planning and tool use [22]–[25], failure detection and failure analysis [21], and automated program

repair [18]. The review established that these mechanisms are typically evaluated by aggregate task-success rate, and that controlled comparison against an unaided resampling baseline is generally absent.

6.2.2 Research Question 2

How do Blind Retry, Reflection-only, and Diagnose+Revise differ in their approach to recovering from failed repository-level coding attempts? The three strategies were implemented as a strictly nested sequence, documented in Sections 4.11 to 4.13 and Table 4.3. Each supplies everything available to its predecessor and adds exactly one further element: Blind Retry supplies nothing about the failure, Reflection-only adds the system's own previous attempt together with a requirement to criticise it, and Diagnose+Revise adds the observed output of the failed test run. Because the shared portion of the prompt is identical in wording across the three, the information supplied after a failure is the only quantity that differs.

6.2.3 Research Question 3

Does the availability of reflection or execution-grounded diagnostic information improve recovery from failed coding attempts? On the evidence of this study, no. Across the logic-class failures, no instance was resolved by any strategy in either round, and in every cell where two or more strategies were evaluated on the same instance the outcomes were identical, including at the level of individual test counts. The instance affording the closest examination returned 6 of its 16 required tests in all six of its recovery cells, spanning three strategies and two rounds. Diagnose+Revise obtained these identical outcomes at approximately three times the input volume of Blind Retry. This conclusion is bounded by the coverage achieved, 16 of 24 cells, and by the limitations recorded in Section 6.5.

6.2.4 Research Question 4

To what extent can observed recovery failures be attributed to reasoning limitations rather than to patch-formatting or patch-application mechanisms? The two are cleanly separable, and separating them changes the conclusion. Of the 8 baseline failures, 4 were format-class, the patch never having been applied, and 4 were logic-class, the patch having applied and the required tests having failed. After the change of patch representation, all 3 evaluable format-class failures were resolved and none of the 4 logic-class failures was. Reporting a single undivided recovery rate would have given 3 recoveries of 7 evaluable instances and appeared to demonstrate that the recovery strategies were effective, when every

recovery was obtained on an instance whose original failure was mechanical. This is the clearest result of the dissertation.

6.2.5 Research Question 5

Are the observed recovery behaviours consistent across repeated evaluation rounds? Wherever a cell was evaluated in both rounds, the outcome was identical, and where sub-test counts are available they too were identical. The qualification is that coverage was incomplete: 16 of the 24 cells defined by the comparison were evaluated and 8 were not, for the resource reasons documented in Section 4.5.2. The supportable claim is therefore consistency across all evaluated cells, stated together with that coverage figure, and not consistency across the failure pool as a whole.

6.3 Key Conclusions

Five conclusions follow from the work.

- Across the logic-class failures evaluated in this study, no measurable difference was found between Blind Retry, Reflection-only, and Diagnose+Revise. All three resolved none of the four such failures, in both rounds, across the 16 of 24 cells that were evaluated.
- Patch-formatting reliability and code-reasoning correctness are separable failure modes. Every format-class baseline failure that could be evaluated was resolved once a programmatically verified patch mechanism was adopted, whereas the logic-class failures remained unresolved under every recovery strategy. Changing how an edit was expressed altered outcomes substantially; changing what the system was told about its own failure did not alter them at all.
- The binary resolution metric used by the benchmark conceals the largest effect observed in this study. The change of patch representation moved one instance from 0 of 16 required tests with 33 regressions to 6 of 16 with none, a movement to which the resolution flag is entirely insensitive, recording the instance as unresolved in both states.
- The null result on logic-class failures suggests that, for this class of defect, the bottleneck is unlikely to be the availability of accurate diagnostic information within a single revision attempt. The convergence of six independent generations on the identical partial solution, under sampling that was not pinned, is more consistent with a stable limit of single-shot code reasoning than with a system whose behaviour is sensitive to post-failure feedback.

- Executing a container-based Linux evaluation harness from a Windows host can silently produce invalid results through line-ending corruption. This is a generalisable methodological caution for coding-agent research conducted on Windows platforms, and is documented in Section 4.9.

6.4 Contributions of the Dissertation

- A fully scripted and auditable empirical pipeline for evaluating failure-recovery strategies on repository-level coding tasks, with every script, prompt, prediction file, and evaluation log published, and all reported results regenerable from those logs by script.
- Empirical evidence, checked across two independent recovery rounds, on the comparative effectiveness of three widely discussed recovery strategies, reported together with the coverage achieved and the cost incurred.
- A demonstration and correction of the confound between patch-formatting reliability and reasoning correctness in coding-agent evaluation, together with the recommendation that patch-application rate be reported separately from task resolution rate.
- A replication, in the previously unexamined setting of repository-level program repair verified by project test suites, of the finding that language models do not reliably correct their own reasoning when given only a request to reconsider [36], [37].
- A documented methodological finding concerning platform-dependent evaluation validity in container-based coding-agent research.
- An observation bearing on benchmark contamination: a provider-side generation refusal on the ground of reproducing memorised training data, obtained independently of the experimenter and recorded in Section 4.18.3, which indicates that benchmark content is present in the model's training corpus.

6.5 Limitations of the Study

The conclusions above are bounded by the following limitations, which are set out in full in Section 5.12 and summarised here.

- Task solvability was not independently audited. A strategy cannot be shown ineffective on a task that no strategy could complete, and a proportion of SWE-bench instances are known to be underspecified or unfairly tested. This is the most serious limitation of the empirical component.

- The logic-class comparison rests on four instances with one observation per cell. No variance estimate is available and no significance test is reported. The study reports that no effect was observed; it does not report that no effect exists.
- Coverage of the comparison was incomplete, 16 of 24 cells having been evaluated, owing to the free-tier request ceiling.
- The task sample is a contiguous prefix of one benchmark drawn from two repositories, 14 of the 20 instances originating from a single project, and is not random.
- Sampling parameters were not fixed, so individual generations are not reproducible although the procedure is.
- A single model from a single provider was used throughout.
- The execution evidence supplied to Diagnose+Revise was truncated to its final 3,000 characters, so that condition was tested with a deliberately narrow feedback window.
- File localisation was supplied from benchmark metadata rather than performed by the system, so the resolution rates reported are not comparable with those of end-to-end agentic systems.

6.6 Scope for Future Work

The limitations above define the work required to establish more than this study does. The items are given in order of priority.

- Audit the solvability of the logic-class instances against their issue descriptions and reference solutions, so that the null result can be attributed with confidence either to the recovery strategies or to the benchmark. Repeating the comparison on SWE-bench Verified, whose instances have been human-screened for specification and test fairness, would achieve the same end.
- Migrate to a provider offering a substantially higher request allowance, so that every cell of the comparison can be evaluated and multiple observations obtained per cell. This single change removes the incomplete-coverage limitation, permits variance estimation and significance testing, and is a precondition for most of the items below.
- Extend the sample beyond 20 instances, by stratified random sampling across the repositories of SWE-bench Lite rather than by taking a contiguous prefix, to test whether the observed null holds at larger scale and outside two projects.

- Widen the evidence window supplied to Diagnose+Revise, and compare windowing strategies against structured extraction of the failing assertion, to determine whether the truncation to 3,000 characters accounts for the absence of any advantage from execution-grounded diagnosis.
- Extend the comparison beyond two recovery rounds, to determine whether iterative diagnosis-based revision eventually succeeds on hard defects given sufficient attempts, and at what cumulative cost.
- Remove the oracle file-localisation simplification by incorporating a repository-search component, so that recovery strategies are tested under unassisted conditions comparable to those of deployed agentic systems.
- Repeat the comparison across several model families and context-window sizes, to establish whether the null result is a property of the strategies or of the particular model evaluated here.
- Extend the strategy ladder beyond the three conditions examined. Candidates suggested by the present findings include constrained or typed decoding to eliminate malformed edits entirely, tool-grounded revision in which the system may execute tests itself rather than receiving a transcript, checkpointing with rollback so that a regression-causing revision can be reverted, and a failure memory carried across attempts.
- Develop and validate recovery-oriented evaluation metrics beyond task-success rate, including diagnosis accuracy, partial-credit sub-test scoring, patch-application rate reported separately, and recovery effort expressed in tokens and wall-clock time. The present study's reliance on sub-test counts to observe its principal effect illustrates the need.
- Screen benchmark instances systematically for training-data contamination, using provider refusal signals of the kind reported in Section 4.18.3 alongside established probing methods, so that reported resolution rates can be interpreted against a known contamination baseline.

Figure 6.1 arranges these strategies as a ladder. The three conditions evaluated in this study vary what the system is told about its failure and produced identical outcomes; the proposed extensions vary what the system is able to do about it. The one intervention that moved outcomes in this study, the change of edit notation, belongs to the second kind.

The Recovery Strategy Ladder

Evaluated in this study: L0 to L2 (solid). Proposed as future work: L3 to L6 (dashed).

L0	Blind Retry	re-attempt, no failure information
L1	Reflection-only	own previous attempt, self-critique
L2	Diagnose+Revise	observed test output, truncated
L3	Constrained decoding	malformed edits impossible by construction
L4	Tool-grounded revision	system executes tests itself
L5	Checkpoint and rollback	revert a regression-causing revision
L6	Failure memory	evidence carried across attempts

L0 to L2 vary what the system is TOLD about its failure, and produced identical outcomes.
L3 to L6 vary what the system can DO about it. The one intervention that moved outcomes in this study, changing the edit notation, belongs to the second kind.

Figure 6.1 The Recovery Strategy Ladder

6.7 Concluding Remarks

This dissertation set out to determine whether giving a repository-level coding system information about its own failure helps it recover. On the evidence obtained, it did not. Neither the system's own reasoning returned to it, nor the observed output of the tests it had failed, changed what it produced; and the strategy supplied with most information cost three times as much as the strategy supplied with none, for outcomes that were identical wherever both were measured.

What did change outcomes was the notation in which an edit was expressed. Moving the clerical work of computing line offsets from the model to a deterministic procedure resolved every format-class failure that could be evaluated, and moved the most demanding

instance in the sample from 0 of 16 required tests with 33 regressions to 6 of 16 with none, without altering the model, the prompt, or the task.

The practical lesson is therefore about measurement before it is about mechanism. A coding system's failures are not of one kind, and an evaluation that does not separate the mechanical from the cognitive will attribute the repair of the former to an improvement in the latter. Had this study reported a single aggregate recovery rate, it would have recorded a plausible-looking 43 per cent and drawn the opposite conclusion from its own data.

The findings are bounded, and Section 6.5 states how narrowly. Four instances, one model, one observation per cell, and an unaudited assumption of task solvability do not settle a general question about self-reflection in coding agents. They do, however, establish that the question cannot be answered by aggregate resolution rates alone, and they indicate where the effort of answering it should be directed.

REFERENCES

- [1] J. He, C. Treude, and D. Lo, “LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 124:1–124:30, 2025.
- [2] H. Li, H. Zhang, and A. E. Hassan, “The rise of AI teammates in software engineering (SE 3.0): How autonomous coding agents are reshaping software engineering,” 2025.
- [3] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 1592–1604.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [5] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An autonomous, LLM-based agent for program repair,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025, pp. 2188–2200.
- [6] X. Wang et al., “OpenHands: An open platform for AI software developers as generalist agents,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [8] A. Madaan, N. Tandon, P. Gupta et al., “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [9] M. Renze and E. Guven, “Self-reflection in LLM agents: Effects on problem-solving performance,” in *First Workshop on Foundational Large Language Models (FLLM)*, 2024.
- [10] X. Chen, M. Lin, N. Scharli, and D. Zhou, “Teaching large language models to self-debug,” in *International Conference on Learning Representations (ICLR)*, 2024.

- [11] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “ExpeL: LLM agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19632–19640.
- [12] X. Bo et al., “Reflective multi-agent collaboration based on large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [13] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, “Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 484–506, 2024.
- [14] Z. Wu et al., “OS-Copilot: Towards generalist computer agents with self-improvement,” *arXiv preprint arXiv:2402.07456*, 2024.
- [15] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic memory for LLM agents,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025.
- [16] Y. Wang and X. Chen, “MIRIX: Multi-agent memory system for LLM-based agents,” *arXiv preprint arXiv:2507.07957*, 2025.
- [17] A. Maharana et al., “Evaluating very long-term conversational memory of LLM agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 13851–13870.
- [18] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, “A systematic literature review on large language models for automated program repair,” *ACM Transactions on Software Engineering and Methodology*, 2026.
- [19] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” in *International Conference on Learning Representations (ICLR)*, 2024.
- [20] X. Liu et al., “AgentBench: Evaluating LLMs as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [21] K. Zhu et al., “Where LLM agents fail and how they can learn from failures,” *arXiv preprint arXiv:2509.25370*, 2025.
- [22] J. Ruan et al., “TPTU: Task planning and tool usage of large language model-based AI agents,” in *Foundation Models for Decision Making Workshop at NeurIPS*, 2023.

- [23] C. H. Song et al., “LLM-Planner: Few-shot grounded planning for embodied agents with large language models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 2998–3007.
- [24] M. Li et al., “API-Bank: A comprehensive benchmark for tool-augmented LLMs,” in *Proceedings of EMNLP*, 2023, pp. 3102–3116.
- [25] Z. Shen, “LLM with tools: A survey,” *arXiv preprint arXiv:2409.18807*, 2024.
- [26] M. Zhuge et al., “Agent-as-a-judge: Evaluate agents with agents,” *arXiv preprint arXiv:2410.10934*, 2024.
- [27] C. Guo et al., “RedCode: Risky code execution and generation benchmark for code agents,” in *NeurIPS Datasets and Benchmarks Track*, 2024.
- [28] T. Masterman, S. Besen, M. Sawtell, and A. Chao, “The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey,” *arXiv preprint arXiv:2404.11584*, 2024.
- [29] A. Bandi, B. Kongari, R. Naguru, S. Pasnoor, and S. V. Vilipala, “The rise of agentic AI: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges,” *Future Internet*, vol. 17, no. 9, p. 404, 2025.
- [30] M. A. Ali, F. Dornaika, and J. Charafeddine, “Agentic AI: A comprehensive survey of architectures, applications, and future directions,” *Artificial Intelligence Review*, vol. 59, p. 11, 2026.
- [31] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [32] R. Sapkota, K. I. Roumeliotis, and M. Karkee, “Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI,” *arXiv preprint arXiv:2505.19443*, 2025.
- [33] M. Robeyns, M. Szummer, and L. Aitchison, “A self-improving coding agent,” *arXiv preprint arXiv:2504.15228*, 2025.
- [34] L. Staufer et al., “The 2025 AI agent index: Documenting technical and safety features of deployed agentic AI systems,” in *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2026, pp. 1536–1576.

- [35] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, “SMART-LLM: Smart multi-agent robot task planning using large language models,” arXiv preprint arXiv:2309.10062, 2024.
- [36] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in International Conference on Learning Representations (ICLR), 2024.
- [37] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, “When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs,” Transactions of the Association for Computational Linguistics, vol. 12, pp. 1417–1440, 2024.